HarvardX Capstone My Own Project - Census Data

Frank Valdivia

2025-06-16

Contents

1 INTRODUCTION

The goal of this project is to build a classification model for the Census Data (1) downloaded from The UCI Machine Learning Repository.

According to the UCI website:

"Extraction was done by Barry Becker from the 1994 Census database. A set of reasonably clean records was extracted using the following conditions: ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HRSWK>0))"

This dataset has 14 columns of which "Income" is the outcome. Income has two levels:

"<=50K" ">50K"

The following is the list of the other features that the dataset has:

- age: continuous.
- workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
- fnlwgt: continuous.
- education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
- education-num: continuous.
- marital-status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.
- occupation: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
- relationship: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
- race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
- sex: Female, Male.
- capital-gain: continuous.
- capital-loss: continuous.
- hours-per-week: continuous.
- native-country: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinadad&Tobago, Peru, Hong, Holand-Netherlands.

+++++++ IMPORTANT +++++++

To run this report, the file adult.zip should be downloaded from the following page and copied to the working directory of the R project running this report.

https://archive.ics.uci.edu/dataset/2/adult

Because processing time to generate the models was significant (more than one day), the six models evaluated were generated and saved as RDS files in GitHub. Download the six models into your working directory:

train_knn.rds: KNN model
train_knn_cv.rds: KNN model with Cross Validation Model
train_dt.rds: Decision Tree model
train_dt_cp.rds: Decision Tree - Complexity Parameter model
train_rf.rds: Random Forest model
train_rf_2.rds: Random Forest wth Cross Validation model

GitHub link:

https://github.com/frankvaldivia/Capstone-FV-DataScience/tree/main

++++++++++++++++++++++++++

Machine learning methods will be used, which means that two sub datasets will be created during this process:

A train set will be generated with the name: censusTrain; this dataset will be used to train the model.

A test set will be generated with the name: censusTest; this dataset will be used to test the model trained using the train dataset.

The output will be a classification model and Accuracy will be used to measure quality of the model.

2 ANALYSIS AND METHODS

The overall process is as follows:

2.1. Loading libraries

2.2. Unzipping donloaded file and Generating datasets

2.3. Analysis and methods

2.1 Loading libraries

To run the report the following R libraries need to be previously installed:

- library(tidyverse)
- library(caret)
- library(ggplot2)
- library(dplyr)
- library(lattice)
- library(gridExtra)
- library(beepr)
- library(dslabs)
- library(rpart)
- library(rpart.plot)
- library(randomForest)
- library(Rborist)
- library(class)
- library(grid)

Above libraries are loaded.

2.2 Unzipping donloaded file and Generating datasets

+++++++ IMPORTANT +++++++

To run this report, the file adult.zip should be downloaded from the following page and copied to the working directory of the R project running this report.

https://archive.ics.uci.edu/dataset/2/adult

Because processing time to generate the models was significant (more than one day), the six models evaluated were generated and saved as RDS files in GitHub. Download the six files (models) into your working directory:

train_knn.rds: KNN model
train_knn_cv.rds: KNN model with Cross Validation Model
train_dt.rds: Decision Tree model
train_dt_cp.rds: Decision Tree - Complexity Parameter model
train_rf.rds: Random Forest model
train_rf_2.rds: Random Forest wth Cross Validation model

GitHub link:

https://github.com/frankvaldivia/Capstone-FV-DataScience/tree/main

++++++++++++++++++++++++++

adult.zip has two files, The author already generated a train dataset and a test dataset. In this project, both files will be put together to create a whole Census_data dataset, which will be used to create a new train dataset and a new test dataset.

Following,

1. Column names are assigned.
2. Empty rows are removed.
3. Temporary train and test data sets are created.

```
beep()

# assigning column names to both temporary files

colnames(census_train) <- c("age", "workclass", "weight", "education",
                            "educationlevel", "maritalstatus", "occupation",
                            "relationship", "race", "sex", "capitalgain",
                            "capitalloss", "hoursperweek", "nativecountry",
                            "income")

colnames(census_test) <- c("age", "workclass", "weight", "education",
                           "educationlevel", "maritalstatus", "occupation",
                           "relationship", "race", "sex", "capitalgain",
                           "capitalloss", "hoursperweek", "nativecountry",
                           "income")

# removing empty rows

temp_train<-census_train[-c(32562,32563),]

temp_test<-census_test[-c(1,16283,16284),]
```

Temporary train and test data sets are created with the following structure and number of rows. These two data sets were originally created by the author and will be used to create a new master census data for this project.

```
##
## Temporary train set: Columns
```

x
age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##
## Temporary train set: Number of Records
```

dim(temp_train)
32561

```
##
## Temporary test set: Columns
```

x
age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##
## Temporary test set: Number of Records
```

dim(temp_test)
16281

```
##
## New Census data: Columns
```

x
age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##
## New Census data: Number of Records
```

dim(census_data)
48842

With the whole new Census Data set, predictors (columns) are redefined as Factors or Integers. The outcome (Y) was redefined as a string with two values: "<=50K" ">50K"

Cleaning Census data continued with the removal of columns that will not be used in the analysis, empty records and the values for the outcome.

The census data set to be used in this project is as follows:

```
##
## Clean Census data: Columns
```

x
age
workclass
education
maritalstatus
occupation
relationship
race
sex
hoursperweek
income

```
##
## clean Census data: Head
```

age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
24	State-gov	Bachelors	Never-married	Adm-clerical	Not-in-family	White	Male	36	<=50K
35	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	6	<=50K
23	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	36	<=50K
38	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	36	<=50K
13	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	36	<=50K
22	Private	Masters	Married-civ-spouse	Exec-managerial	Wife	White	Female	36	<=50K

```
##
## clean Census data: Number of Records
```

dim(census_data)
48842

```
##
## clean Census data: levels for every predictor and outcome(Income)
## $age
## NULL
##
## $workclass
## [1] "?"                "Federal-gov"      "Local-gov"        "Never-worked"
## [5] "Private"          "Self-emp-inc"     "Self-emp-not-inc" "State-gov"
## [9] "Without-pay"
##
## $education
##  [1] "Preschool"    "1st-4th"      "5th-6th"      "7th-8th"      "9th"
##  [6] "10th"         "11th"         "12th"         "HS-grad"      "Assoc-acdm"
## [11] "Assoc-voc"    "Some-college" "Bachelors"    "Masters"      "Prof-school"
## [16] "Doctorate"
##
## $maritalstatus
## [1] "Divorced"              "Married-AF-spouse"     "Married-civ-spouse"
## [4] "Married-spouse-absent" "Never-married"         "Separated"
## [7] "Widowed"
##
## $occupation
##  [1] "?"                 "Adm-clerical"      "Armed-Forces"
##  [4] "Craft-repair"      "Exec-managerial"   "Farming-fishing"
##  [7] "Handlers-cleaners" "Machine-op-inspct" "Other-service"
## [10] "Priv-house-serv"   "Prof-specialty"    "Protective-serv"
## [13] "Sales"             "Tech-support"      "Transport-moving"
##
## $relationship
```

```
## [1] "Husband"        "Wife"           "Unmarried"      "Not-in-family"
## [5] "Other-relative" "Own-child"
##
## $race
## [1] "Amer-Indian-Eskimo" "Asian-Pac-Islander" "Black"
## [4] "Other"              "White"
##
## $sex
## [1] "Female" "Male"
##
## $hoursperweek
## NULL
##
## $income
## [1] "<=50K" ">50K"
```

2.3 Analysis and methods

First, exploratory analysis will be carried out showing frequencies for predictors and the outcome (Income).

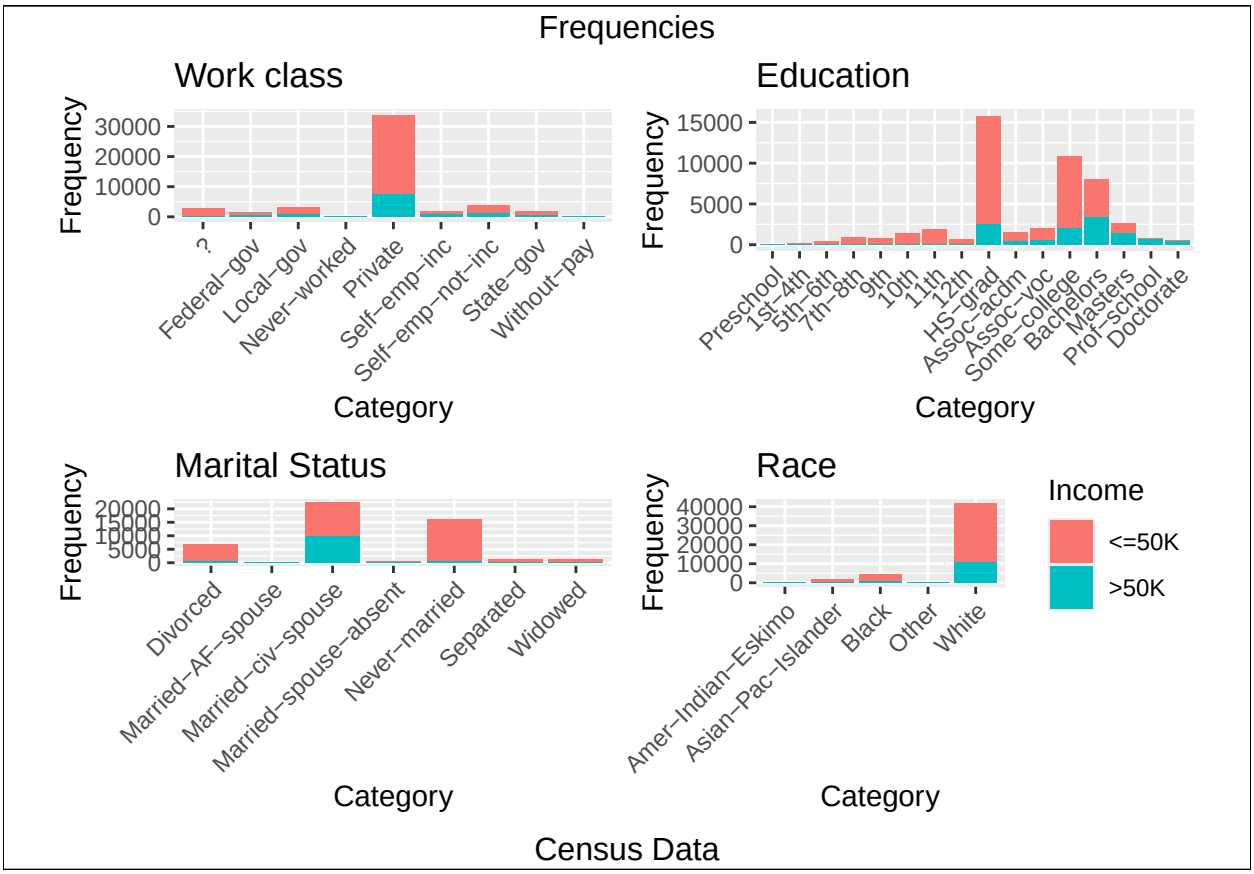
Then, a description of the measure of success (accuracy) will be included before elaborating on the models applied to Census data.

Every model will include specific results and plots and all will have their Accuracy estimated and compared.

2.3.1 Exploratory Analysis

Following are the frequencies for the following predictors. They are all categories:

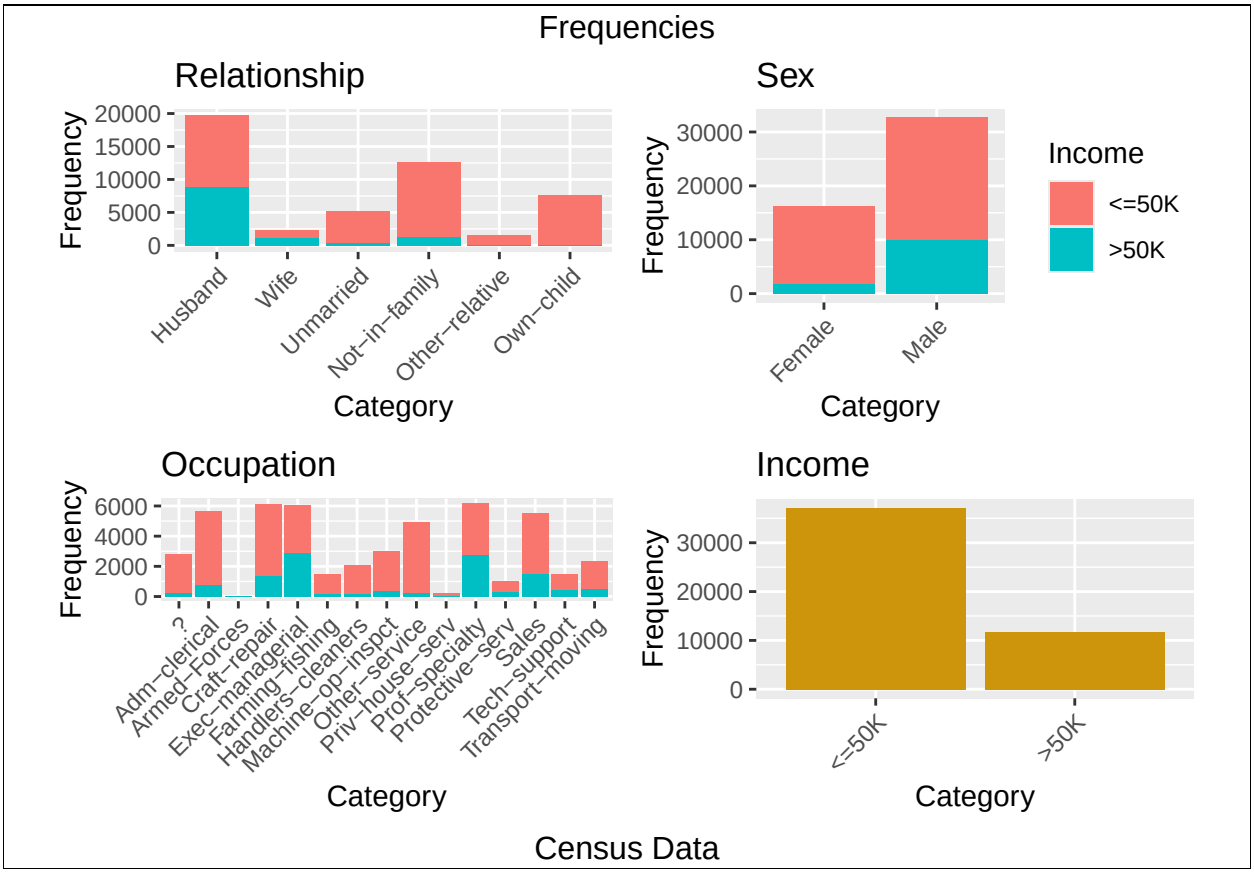
- Work class
- Education
- Marital Status
- Race



Previous plots suggest that there is some level of dependancy of income on some specific categories, i.e. HS-Grad, Bachelors, Some College in Education, or Private in Work class.

Following are the frequencies for the following predictors. They are all categories:

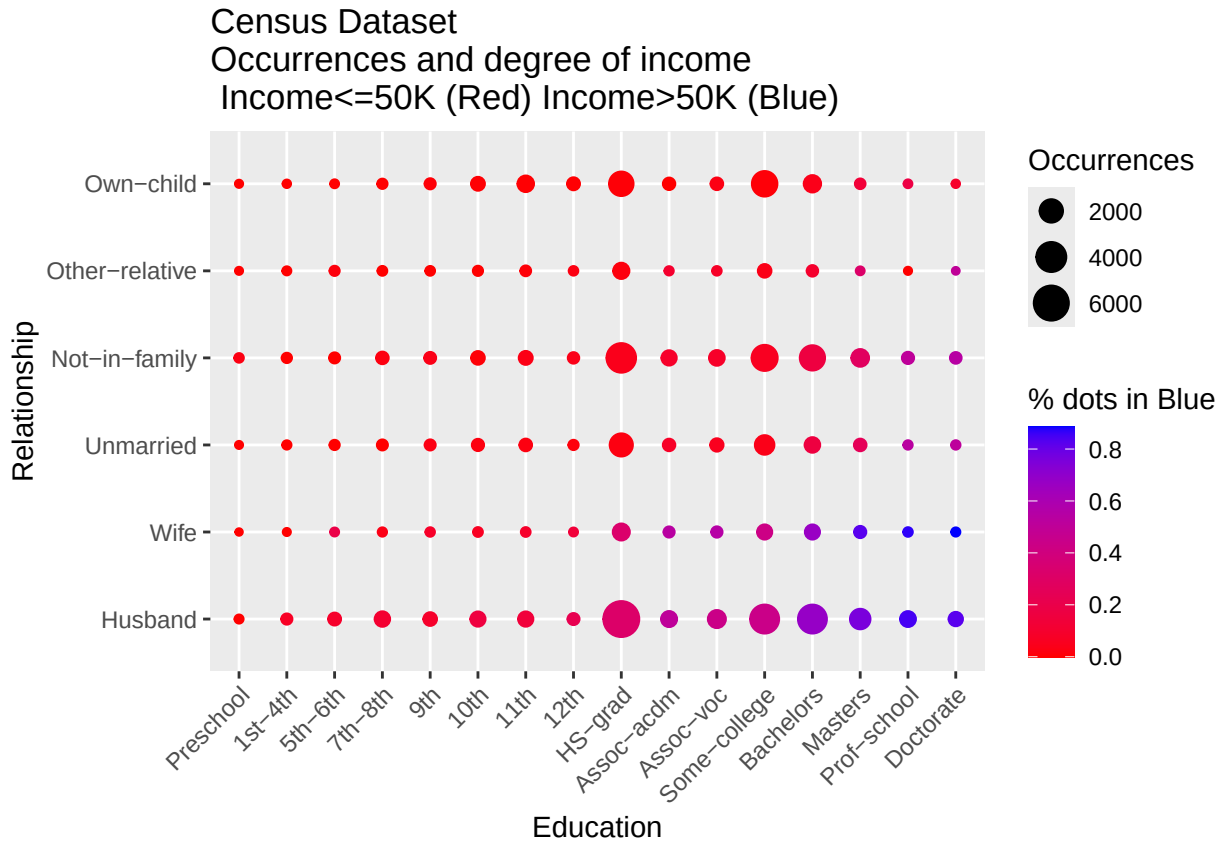
- Relationship
- Sex
- Occupation
- Income



Previous plots suggest that there is some level of dependancy of income on some specific categories, i.e. Female in Sex, or Own-child in Relationshp.

The following graph shows the values of Income for the whole Census dataset. Because this is a two dimensional graph, only two predictors out of the nine are used, Education and Relationship.

Every dot shows the frequency of occurrences that have ">50K" as income in blue as a percentage of the total number of records for that combination of Education and Relationship. The size of the dot represents the number of records for that combination of Education and Relationship.



2.3.2 Accuracy and the Confusion Matrix

The accuracy of every model will be calculated and compared. Accuracy is the percentage of correct Income predictions on the Census data set.

$$Accuracy = \frac{RightPredictions}{TotalPredictions}$$

The confusion Matrix will also be reported for every model. The confusion Matrix from R summarizes the performance of the model. It compares the predicted values against the test values.

To run the models and get the confusin matrix, two sets were created from the Census data set. The train set and the test set, which is 30% of the whole Census data set.

Following are the structure and number of rows that each of these datasets has:

```
##
## censusTrain: Head
```

age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
24	State-gov	Bachelors	Never-married	Adm-clerical	Not-in-family	White	Male	36	<=50K
35	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	6	<=50K
23	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	36	<=50K
38	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	36	<=50K
13	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	36	<=50K
22	Private	Masters	Married-civ-spouse	Exec-managerial	Wife	White	Female	36	<=50K

```
##
## censusTrain: Number of Records
```

dim(censusTrain)
34190

```
##
## censusTest: Head
```

	age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
7	34	Private	9th	Married-spouse-absent	Other-service	Not-in-family	Black	Female	9	<=50K
8	37	Self-emp-not-inc	HS-grad	Married-civ-spouse	Exec-managerial	Husband	White	Male	41	>50K
27	4	Private	HS-grad	Never-married	Craft-repair	Own-child	White	Male	36	<=50K
33	30	Private	Bachelors	Divorced	Exec-managerial	Own-child	White	Male	36	<=50K
39	16	Private	Some-college	Married-civ-spouse	Sales	Husband	White	Male	33	>50K

	age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
42	38	Self-emp-not-inc	Bachelors	Married-civ-spouse	Prof-specialty	Husband	White	Male	36	<=50K

```
##
## censusTest: Number of Records
```

dim(censusTest)
14652

2.3.3 Models

The following models will be applied to the Census train data set and:

1. Predictions will be estimated
2. Plots will be generated
3. The confusion matrix will be produced comparing the predictions on the test set to the actual values of the test set.

Six models will be built and each model assessed against its Accuracy.

1. KNN (K-Nearest Neighbors): It classifies data points based on their similarity to existing data points. The similarity is measured thorugh the euclidian distance and the minimum distance is chosen to predict the values. During the running, this model was saved in a file (train_knn.rds) and uploaded to GitHub.
2. KNN with Cross Validation Model: It is a KNN model that uses tunning to find the optimum value of K. During the running, this model was saved in a file (train_knn_cv.rds) and uploaded to GitHub.
3. Decision Tree: It splits data into subsets based on a series of conditions identified based on minimizing a coefficient. Each internal node represents a test (Yes or No) on a preditor, each branch represents the outcome of the test, and each leaf node represents the predicted value "<=50K" or ">50K". During the running, this model was saved in a file (train_dt.rds) and uploaded to GitHub.
4. Decision Tree - Complexity Parameter: The complexity parameter optimizes the size and complexity of the tree during growth and can improve generalization and avoid overfitting. During the running, this model was saved in a file (train_dt_cp.rds) and uploaded to GitHub.
5. Random Forest: It randomly generates many decision trees from the same data set to strenghten the prediction. During the running, this model was saved in a file (train_rf.rds) and uploaded to GitHub.
6. Random Forest with Cross Validation: It applies cross validation to the Random Forst model. During the running, this model was saved in a file (train_rf_2.rds) and uploaded to GitHub.

2.3.3.1 First model: KNN

The first model applied is KNN (K-Nearest Neighbors), which classifies data points based on their similarity to existing data points. The similarity is measured through the euclidian distance and the minimum distance is chosen to select the neighborhood and predict the values.

During the running, this model was saved in a file (train_knn.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

```
# KNN ==========================================================================

beep()

# The code in comments might take from minutes to hours to run

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:55:27 CDT"
```

```
set.seed(6)

# train_knn <-  train(income ~ .,
#                     method = "knn",
#                     data = censusTrain,
#                     tuneGrid = data.frame(k = seq(3, 7, 2)))
# saveRDS(train_knn,"train_knn.rds")

# Loading from file to save running time
train_knn <- readRDS("train_knn.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:55:27 CDT"
```

The following is the resulting KNN model. It shows that accuracy is maximum at K = 21, with an Accuracy of 0.8047201

```
## k-Nearest Neighbors
##
## 34190 samples
##     9 predictor
##     2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 34190, 34190, 34190, 34190, 34190, 34190, ...
## Resampling results across tuning parameters:
##
##   k   Accuracy   Kappa
##    3  0.7836537  0.4024940
##    5  0.7925305  0.4191783
##    7  0.7972746  0.4268867
##    9  0.8001057  0.4309620
##   11  0.8023641  0.4340998
##   13  0.8032208  0.4341401
```

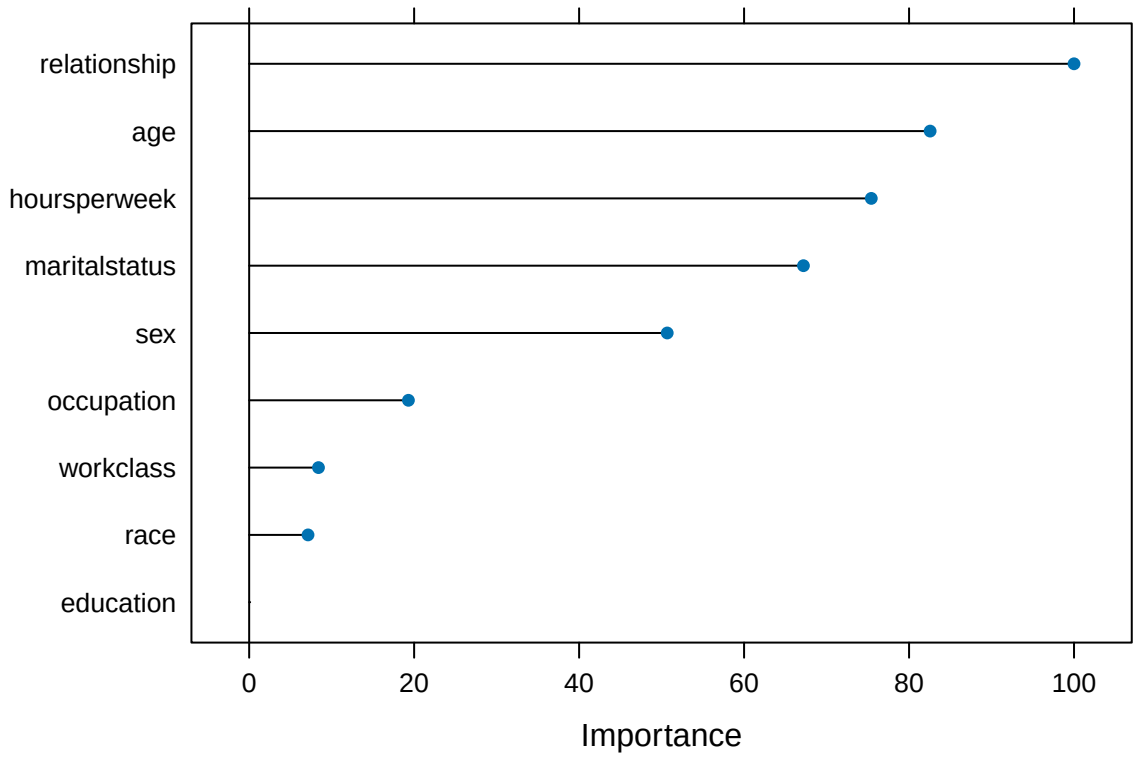
```
##   15  0.8037427  0.4338297
##   17  0.8038641  0.4324673
##   19  0.8041744  0.4321608
##   21  0.8047201  0.4327572
##   23  0.8044929  0.4308773
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 21.
```

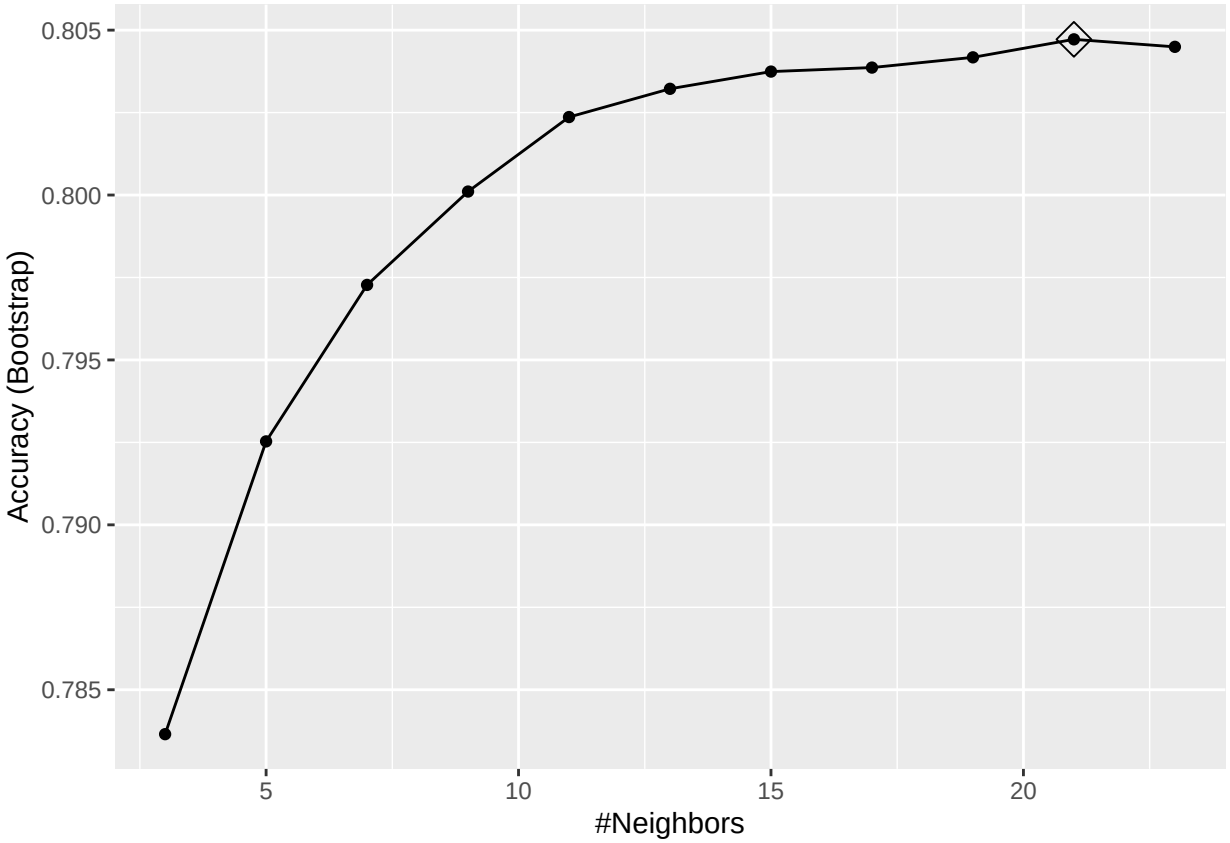
Following is the Variables Importance plot for the KNN model. This graph shows how much each predictor variable contributes to the model's predictive power.

It shows that :
- relationship
- age
- hoursperweek

Are the most important variables for this model.

Next is a plot of Accuracy versus K showing that Accuracy is maximized at K = 21.





The following section shows the Confusion Matrix for this model. It shows:

- Accuracy : 0.81
- Sensitivity : 0.9031
- Specificity : 0.5140
- Balanced Accuracy : 0.7085

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K  >50K
##      <=50K 10066  1704
##      >50K   1080  1802
##
##                Accuracy : 0.81
##                  95% CI : (0.8035, 0.8163)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4442
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9031
##             Specificity : 0.5140
##          Pos Pred Value : 0.8552
```

```
##          Neg Pred Value : 0.6253
##              Prevalence : 0.7607
##          Detection Rate : 0.6870
##    Detection Prevalence : 0.8033
##       Balanced Accuracy : 0.7085
##
##        'Positive' Class : <=50K
##
```

The First model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21

2.3.3.2 Second model: KNN with Cross Validation

KNN with Cross Validation is a model that uses tunning to find the optimum value of K.

During the running, this model was saved in a file (train_knn_cv.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

```
# KNN Cross Validation ========================================

beep()

# The code in comments might take from minutes to hours to run

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:56:33 CDT"
```

```
# control <- trainControl(method = "cv", number = 5, p = .9)
# train_knn_cv <- train(income ~ ., method = "knn",
#                       data = censusTrain,
#                       tuneGrid = data.frame(k = seq(3, 7, 2)),
#                       trControl = control)
# saveRDS(train_knn_cv,"train_knn_cv.rds")

# Loading model from file to save running time
train_knn_cv <- readRDS("train_knn_cv.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:56:33 CDT"
```

The following is the resulting KNN_CV model. It shows that accuracy is maximum at K = 13

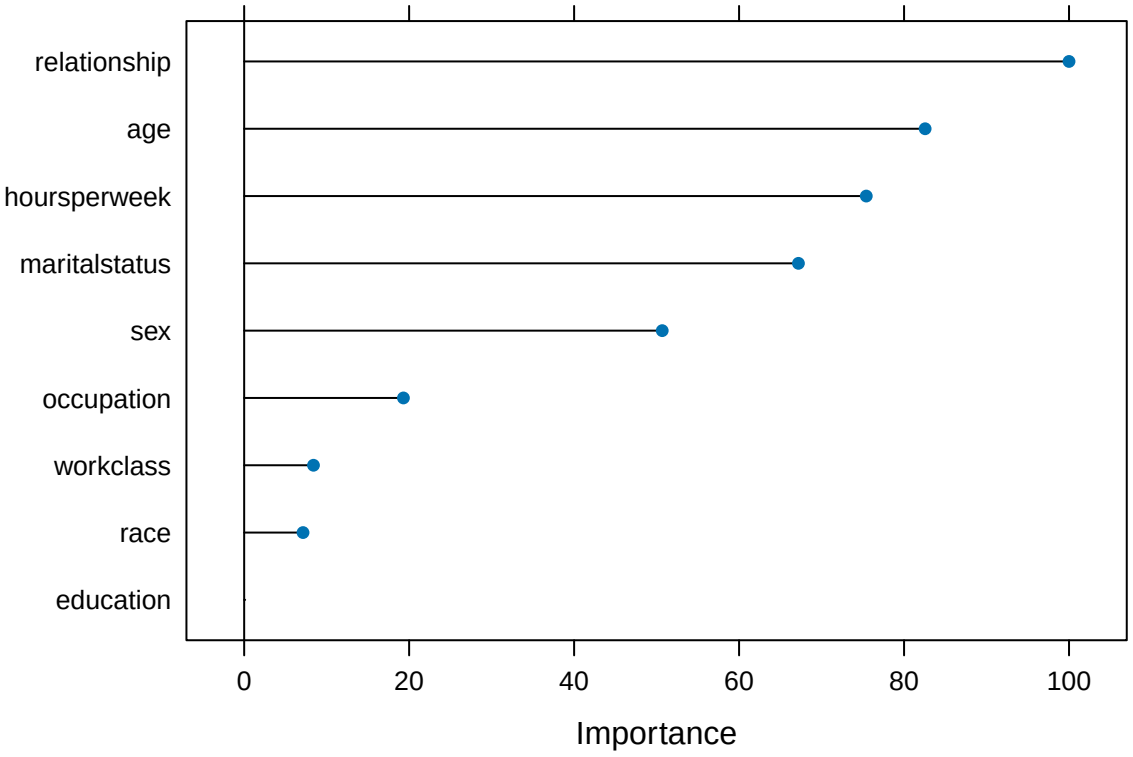
```
## k-Nearest Neighbors
##
## 34190 samples
##     9 predictor
##     2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 27351, 27352, 27353, 27352, 27352
## Resampling results across tuning parameters:
##
##   k   Accuracy   Kappa
##    3  0.8030711  0.4455680
##    5  0.8059667  0.4487606
##    7  0.8076046  0.4491882
##    9  0.8090961  0.4505912
##   11  0.8088621  0.4472322
##   13  0.8103538  0.4492450
##   15  0.8095934  0.4448671
##   17  0.8088329  0.4420376
##   19  0.8088329  0.4411183
##   21  0.8086866  0.4398318
```

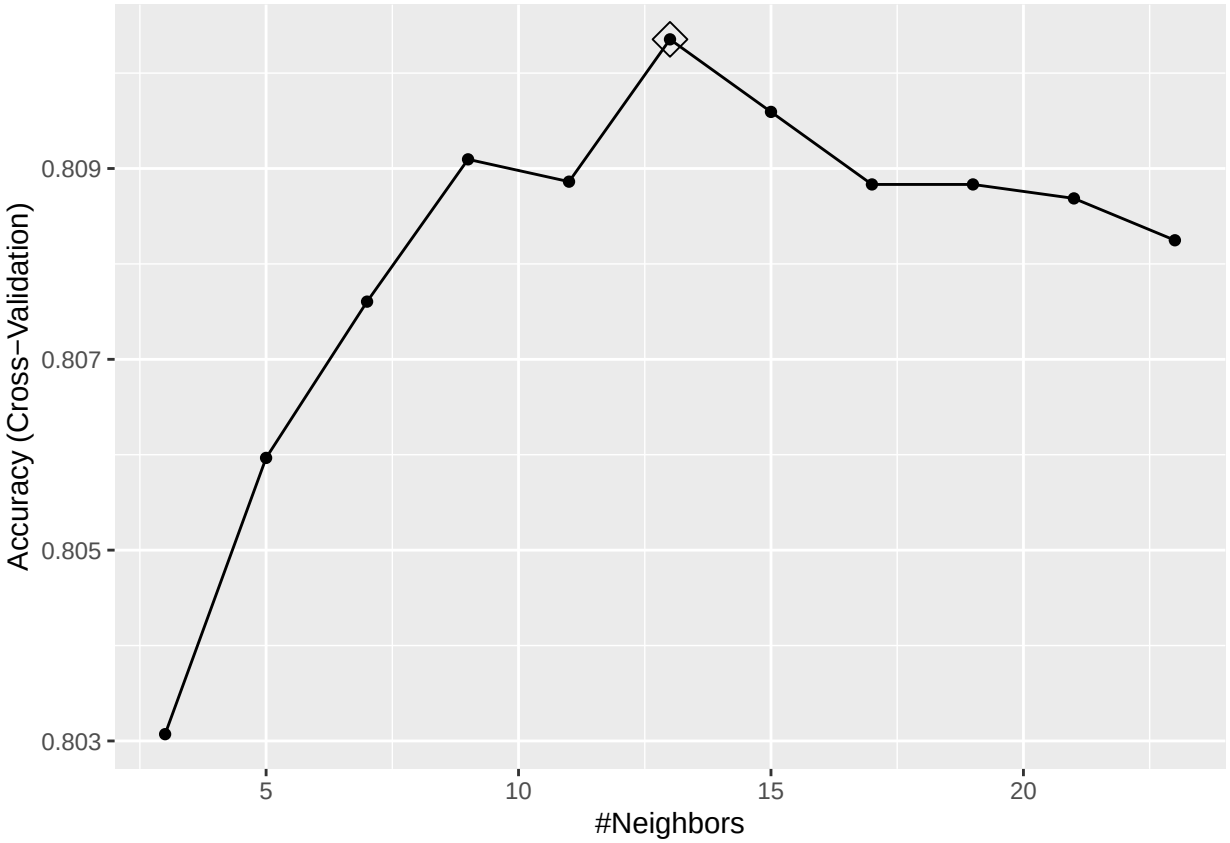
```
##   23  0.8082479  0.4380242
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 13.
```

Following is the Variables Importance plot for the KNN_CV model. This graph shows how much each predictor variable contributes to the model's predictive power.

It shows that :
- relationship
- age
- hoursperweek

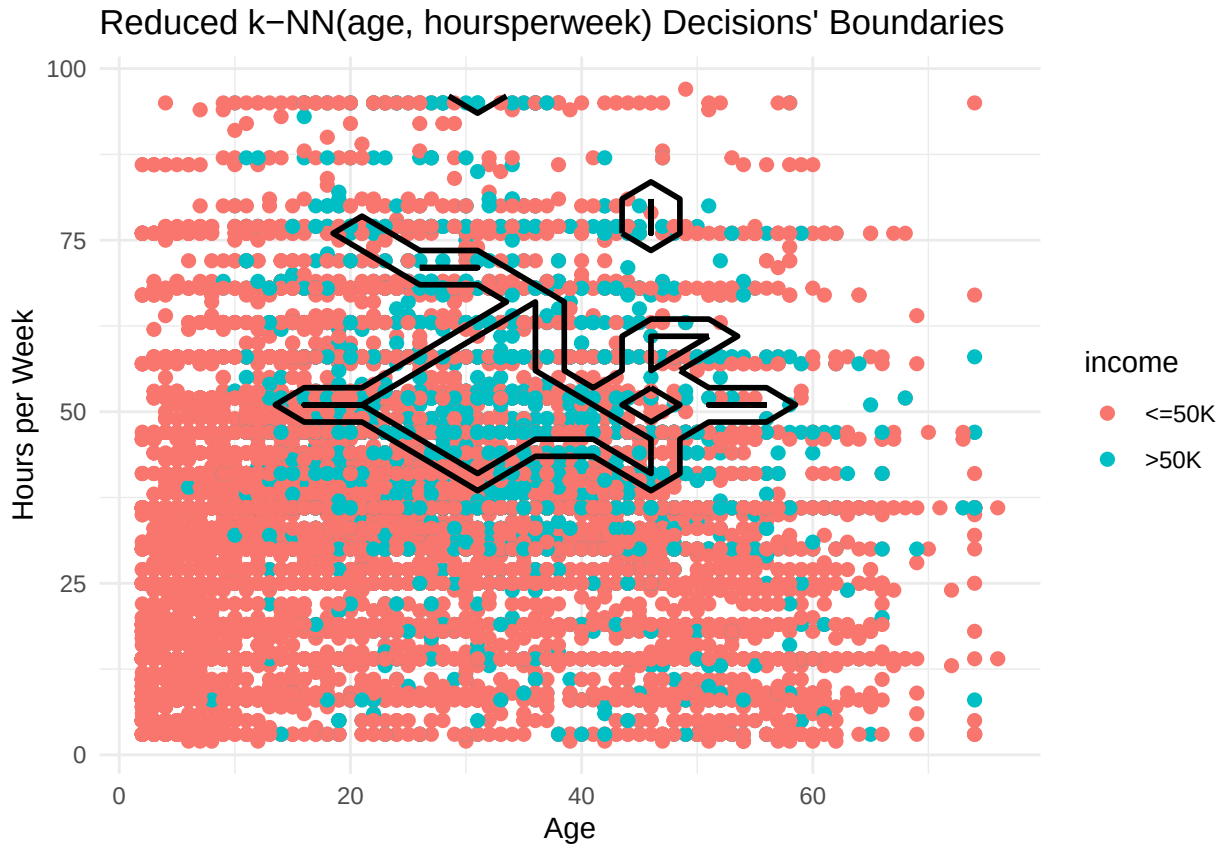
Are the most important variables for this model.





The previous plot shows Accuracy versus K showing that Accuracy is maximized at K = 13.

The following plot is illustrative of the behavior of the KNN model. Because there are 9 predictors, they cannot be plotted in just one graph with two dimensions. A reduced KNN model was generated with just the two most important continuous variables for KNN: "Age" and "Hours per Week". The contour of the reduced model's predictions was then plotted against CensusTrain. The contour shows the decisions' boundaries to assign "<=50K" or ">50K"



The following section shows the Confusion Matrix for this model. It shows:

- Accuracy : 0.8101
- Sensitivity : 0.8980
- Specificity : 0.5308
- Balanced Accuracy : 0.7144

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K  >50K
##      <=50K 10009  1645
##      >50K   1137  1861
##
##                Accuracy : 0.8101
##                  95% CI : (0.8037, 0.8165)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4512
##
```

```
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.8980
##             Specificity : 0.5308
##          Pos Pred Value : 0.8588
##          Neg Pred Value : 0.6207
##              Prevalence : 0.7607
##          Detection Rate : 0.6831
##    Detection Prevalence : 0.7954
##       Balanced Accuracy : 0.7144
##
##        'Positive' Class : <=50K
##
```

The second model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13

2.3.3.3 Third model: Decision Tree

Decision Tree splits data into subsets based on a series of conditions identified based on minimizing a coefficient. Each internal node represents a test (Yes or No) on a predictor, each branch represents the outcome of the test, and each leaf node represents the predicted value "<=50K" or ">50K".

During the running, this model was saved in a file (train_dt.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

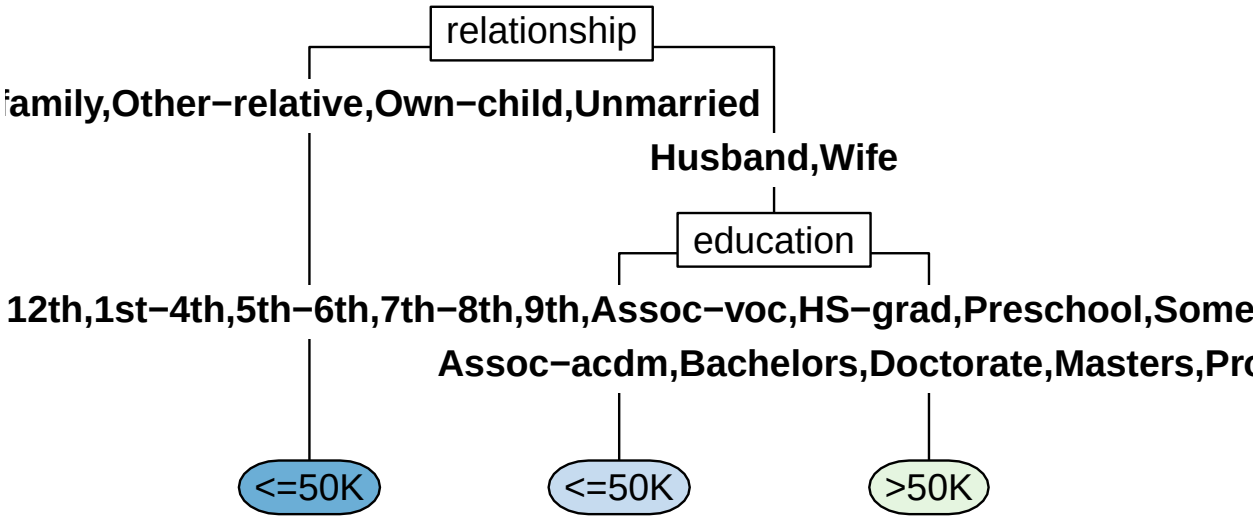
```
# DECISION TREE ============================================================

beep()

# The code in comments should run relatively fast in comparison to previous models

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:40 CDT"
```

```
# train_dt <- rpart(income ~ ., data = censusTrain)
# saveRDS(train_dt,"train_dt.rds")

# Loading model from file to save running time
train_dt <- readRDS("train_dt.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:40 CDT"
```

Next is the plot of the Decision Tree model followed by the rules of the model and its Accuracy, which is maximum at 0.8187




```
## n= 34190
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
## 1) root 34190 8181 <=50K (0.76071951 0.23928049)
##   2) relationship=Not-in-family,Other-relative,Own-child,Unmarried 18822 1227 <=50K (0.93481033 0.065
##   3) relationship=Husband,Wife 15368 6954 <=50K (0.54750130 0.45249870)
##     6) education=10th,11th,12th,1st-4th,5th-6th,7th-8th,9th,Assoc-voc,HS-grad,Preschool,Some-college
##     7) education=Assoc-acdm,Bachelors,Doctorate,Masters,Prof-school 5125 1517 >50K (0.29600000 0.704
```

Following is the Variables Importance list for the Decision Tree model. This list shows how much each predictor variable contributes to the model's predictive power.

It shows that :
- relationship
- maritalstatus
- education

Are the most important variables for this model.

	Overall
relationship	2538.2200
maritalstatus	2502.5749
education	2281.4666
occupation	2101.1456
age	1213.4730
hoursperweek	212.3358
workclass	154.1901
race	0.0000
sex	0.0000

The following section shows the Confusion Matrix for this model. It shows:

- Accuracy : 0.8187
- Sensitivity : 0.9402
- Specificity : 0.4324
- Balanced Accuracy : 0.6863

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K  >50K
##      <=50K 10480  1990
##      >50K    666  1516
##
##                Accuracy : 0.8187
##                  95% CI : (0.8124, 0.8249)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.428
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9402
##             Specificity : 0.4324
##          Pos Pred Value : 0.8404
##          Neg Pred Value : 0.6948
##              Prevalence : 0.7607
##          Detection Rate : 0.7153
##    Detection Prevalence : 0.8511
##       Balanced Accuracy : 0.6863
##
##        'Positive' Class : <=50K
##
```

The third model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13
3: Decision tree	0.8187278	

2.3.3.4 Fourth model: Decision Tree with Complexity Parameter

The complexity parameter in a Decision Tree optimizes the size and complexity of the tree during growth and can improve generalization and avoid overfitting.

During the running, this model was saved in a file (train_dt_cp.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

```
# Decision Tree with Complexity Parameter (CP) ========================================

beep()

# The code in comments might take from minutes to hours to run

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:41 CDT"
```

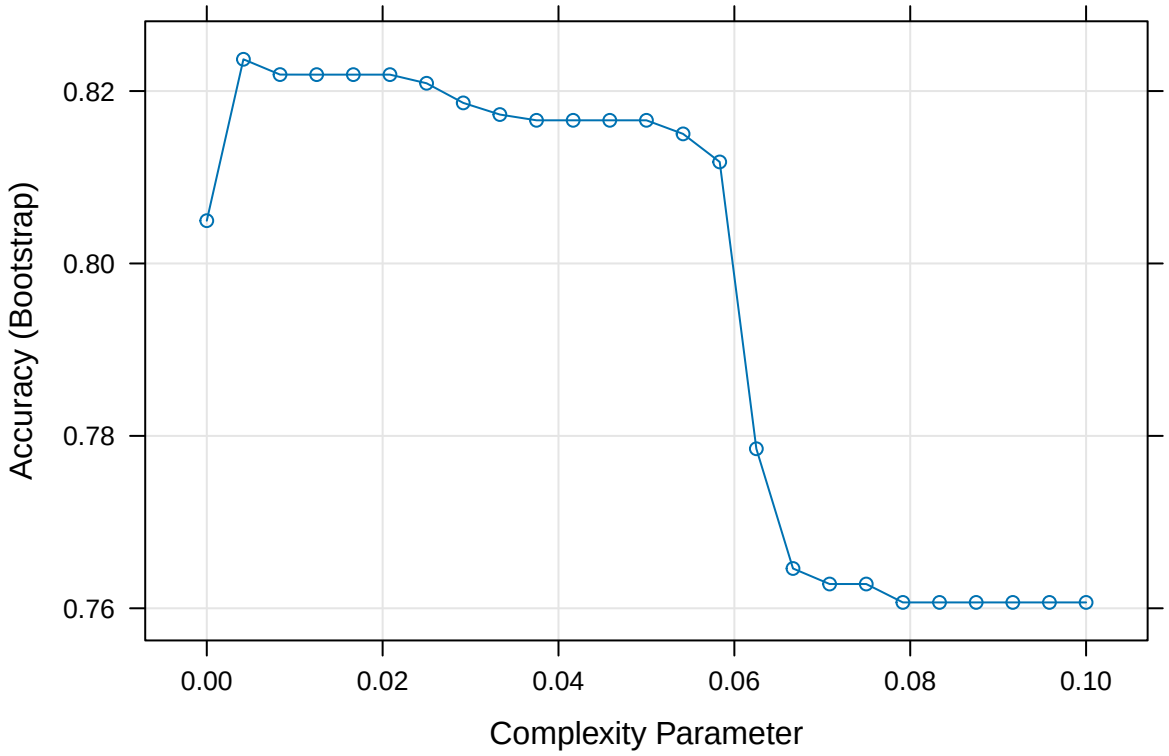
```
# train_dt_cp <- train(income ~ .,
#                      method = "rpart",
#                      tuneGrid = data.frame(cp = seq(0.0, 0.1, len = 25)),
#                      data = censusTrain)
# saveRDS(train_dt_cp,"train_dt_cp.rds")

# Loading model from file to save running time
train_dt_cp <- readRDS("train_dt_cp.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:43 CDT"
```

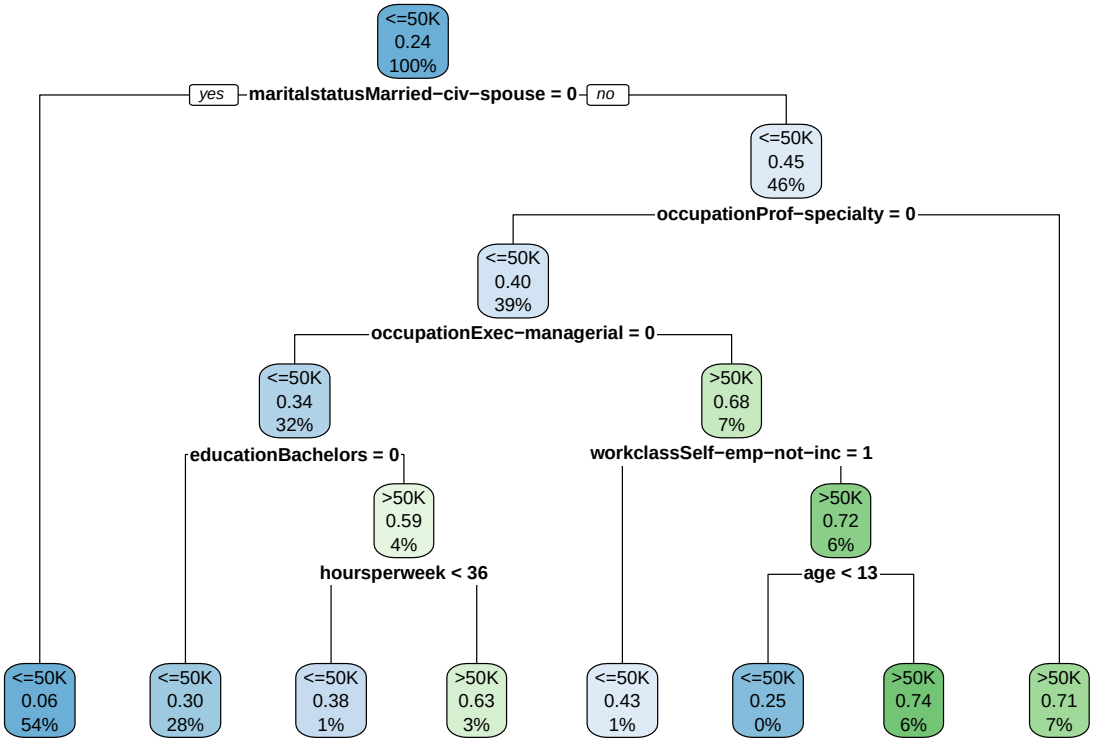
Next is the Decision Tree model and the plot of Complexity Parameter versus Accuracy. It shows that Accuracy is maximum (0.8236853) for a CP of 0.004166667

```
## CART
##
## 34190 samples
##     9 predictor
##     2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 34190, 34190, 34190, 34190, 34190, 34190, ...
## Resampling results across tuning parameters:
##
##   cp           Accuracy   Kappa
##   0.000000000  0.8049633  0.44798176
##   0.004166667  0.8236853  0.46033632
##   0.008333333  0.8219124  0.46121213
##   0.012500000  0.8219057  0.46254003
##   0.016666667  0.8219057  0.46254003
##   0.020833333  0.8219057  0.46254003
##   0.025000000  0.8209024  0.45534424
##   0.029166667  0.8186261  0.43819817
##   0.033333333  0.8172752  0.42720923
```

```
##   0.037500000  0.8166055  0.42072957
##   0.041666667  0.8166055  0.42072957
##   0.045833333  0.8166055  0.42072957
##   0.050000000  0.8166055  0.42072957
##   0.054166667  0.8150313  0.41025654
##   0.058333333  0.8117781  0.38711005
##   0.062500000  0.7785039  0.13890877
##   0.066666667  0.7646138  0.03116930
##   0.070833333  0.7628152  0.01625266
##   0.075000000  0.7628152  0.01625266
##   0.079166667  0.7606807  0.00000000
##   0.083333333  0.7606807  0.00000000
##   0.087500000  0.7606807  0.00000000
##   0.091666667  0.7606807  0.00000000
##   0.095833333  0.7606807  0.00000000
##   0.100000000  0.7606807  0.00000000
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.004166667.
```

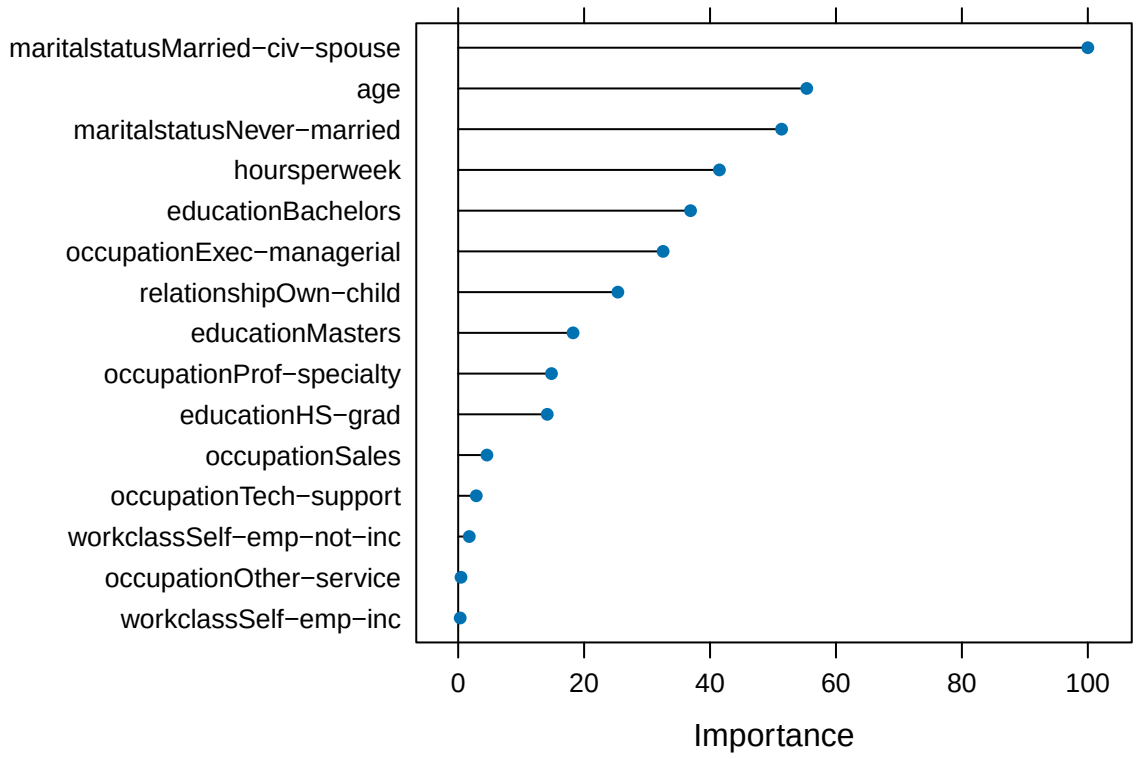


Following are the plot of the Decision Tree model with Complexity Parameter and the description of its rules.




```
## n= 34190
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##  1) root 34190 8181 <=50K (0.7607195 0.2392805)
##    2) maritalstatusMarried-civ-spouse< 0.5 18593 1195 <=50K (0.9357285 0.0642715) *
##    3) maritalstatusMarried-civ-spouse>=0.5 15597 6986 <=50K (0.5520934 0.4479066)
##      6) occupationProf-specialty< 0.5 13357 5387 <=50K (0.5966909 0.4033091)
##       12) occupationExec-managerial< 0.5 10871 3686 <=50K (0.6609328 0.3390672)
##         24) educationBachelors< 0.5 9576 2916 <=50K (0.6954887 0.3045113) *
##         25) educationBachelors>=0.5 1295  525 >50K (0.4054054 0.5945946)
##           50) hoursperweek< 35.5 182   70 <=50K (0.6153846 0.3846154) *
##           51) hoursperweek>=35.5 1113  413 >50K (0.3710692 0.6289308) *
##       13) occupationExec-managerial>=0.5 2486  785 >50K (0.3157683 0.6842317)
##         26) workclassSelf-emp-not-inc>=0.5 291  124 <=50K (0.5738832 0.4261168) *
##         27) workclassSelf-emp-not-inc< 0.5 2195  618 >50K (0.2815490 0.7184510)
##           54) age< 12.5 84   21 <=50K (0.7500000 0.2500000) *
##           55) age>=12.5 2111  555 >50K (0.2629086 0.7370914) *
##      7) occupationProf-specialty>=0.5 2240  641 >50K (0.2861607 0.7138393) *
```

The Variables Importance plot is shown for the Decision Tree model. This plot shows how much each variable contributes to the model's predictive power.



The following section shows the Confusion Matrix for this model. It shows:

- Accuracy : 0.8232
- Sensitivity : 0.9351
- Specificity : 0.4675
- Balanced Accuracy : 0.7013

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K  >50K
##      <=50K 10423  1867
##      >50K    723  1639
##
##                Accuracy : 0.8232
##                  95% CI : (0.817, 0.8294)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4533
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
```

```
##             Sensitivity : 0.9351
##             Specificity : 0.4675
##          Pos Pred Value : 0.8481
##          Neg Pred Value : 0.6939
##              Prevalence : 0.7607
##          Detection Rate : 0.7114
##    Detection Prevalence : 0.8388
##       Balanced Accuracy : 0.7013
##
##        'Positive' Class : <=50K
##
```

The fourth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13
3: Decision tree	0.8187278	
4: Decision tree with CP (Complexity Parameter)	0.8232323	cp = 0.00416666666666667

2.3.3.5 Fifth model: Random Forest

Random Forest generates many decision trees from the same data set to strenghten the prediction.

During the running, this model was saved in a file (train_rf.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

```
# Random Forest ========================================

beep()

# The code in comments might take from minutes to hours to run

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:44 CDT"
```

```
set.seed(6)
# For the generation of this report, the randomForest function was used
# train_rf<-randomForest(formula=income~.,
#                              data=censusTrain,
#                              ntree=500,
#                             do.trace=TRUE)
# saveRDS(train_rf,"train_rf.rds")

# Loading model from file to save running time
train_rf <- readRDS("train_rf.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 09:57:45 CDT"
```

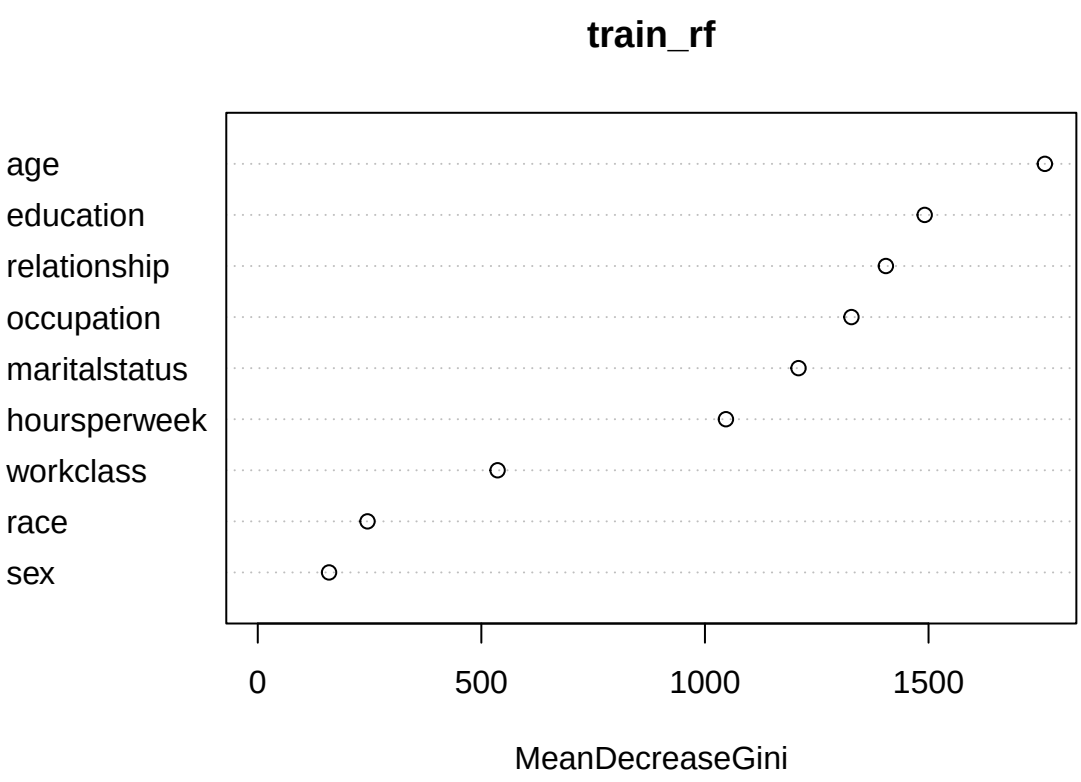
Next is the Random Forest model using 500 trees with 3 variables tried at each split.

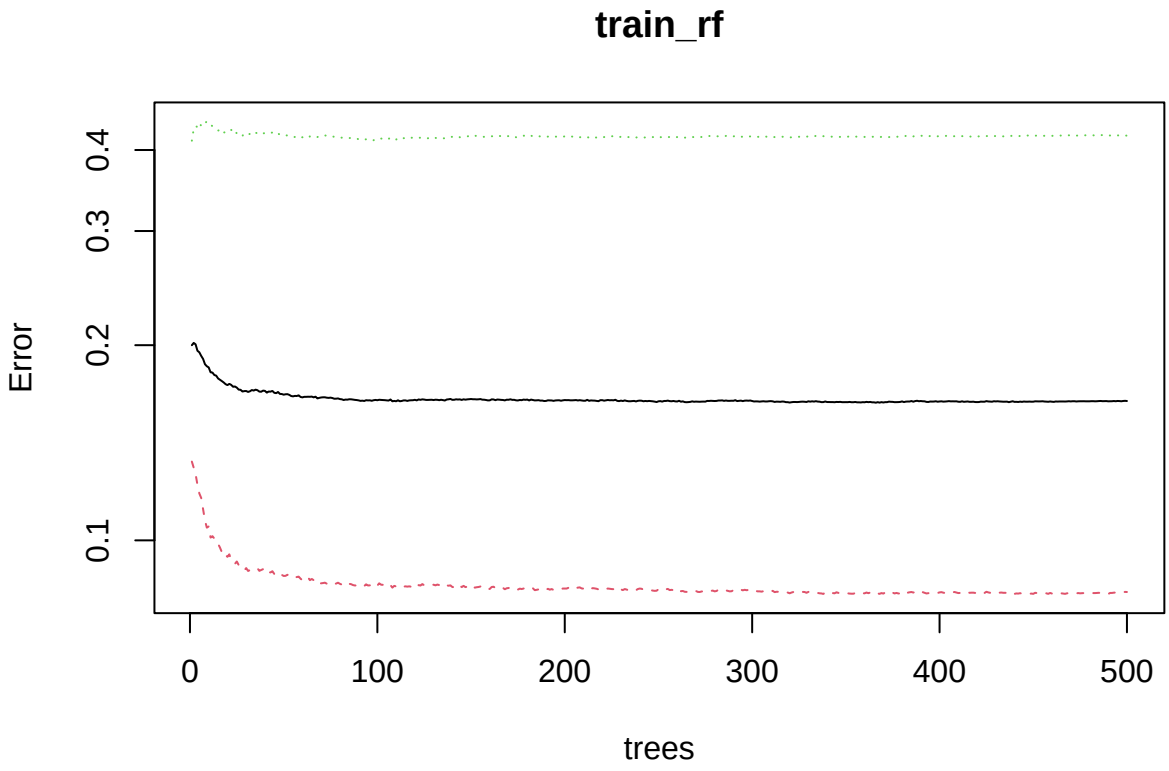
```
##
## Call:
##  randomForest(formula = income ~ ., data = censusTrain, ntree = 500,      do.trace = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 16.4%
## Confusion matrix:
##       <=50K >50K class.error
## <=50K 23844 2165  0.08324042
## >50K   3443 4738  0.42085320
```

The variables Importance plot is shown for the Random Forest model. This plot shows how much each variable contributes to the models's predictive power. It shows that Age, Education and Relationship are the most important variables for the model. This is also reflected in the plot of Variables versus MeanDecreaseGini that follows.

	Overall
age	1760.1372
education	1491.3574
relationship	1404.5718
occupation	1327.5534

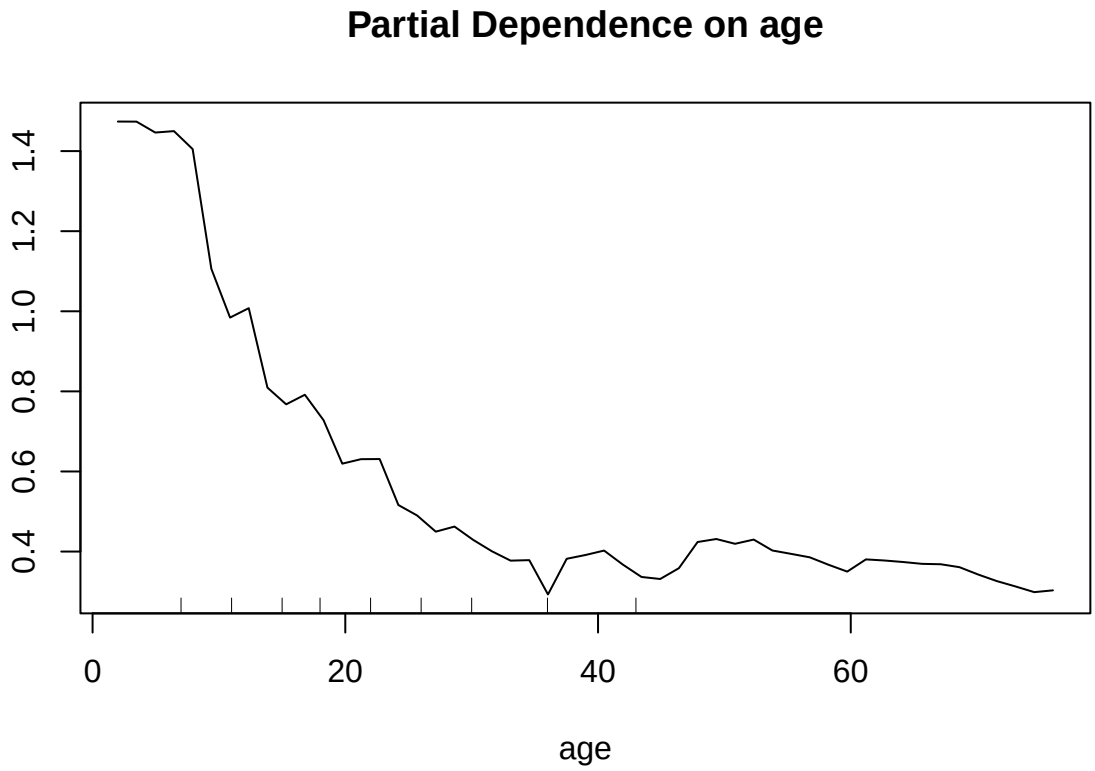
	Overall
maritalstatus	1209.2397
hoursperweek	1046.8976
workclass	536.3182
race	245.3967
sex	159.4219

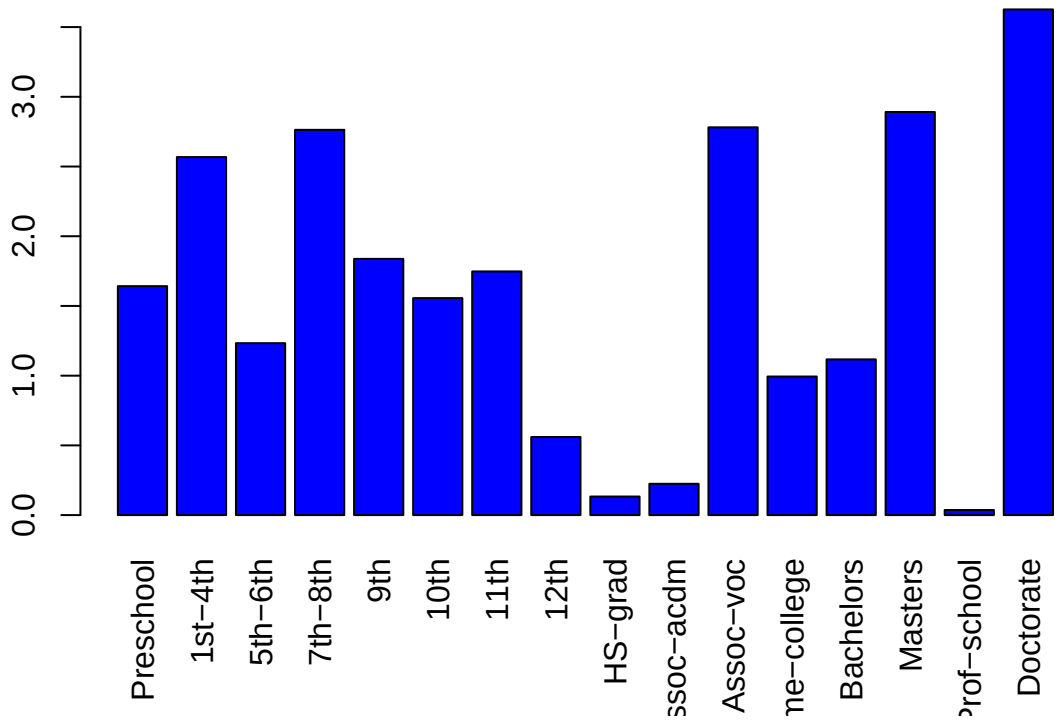




The previous plot shows how error is decreasing as number of trees is increasing

The following two graphs show the dependance of the model on the values of the two most important variables: age and education.





It shows that the model is more dependant for Ages less than 25 and Education as Doctorate and Masters.

The following section shows the Confusion Matrix for this model. It shows:

- Accuracy: 0.7497
- Sensitivity: 0.8271
- Specificity: 0.5037
- Balanced Accuracy: 0.8412

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##      <=50K  9219 1740
##      >50K   1927 1766
##
##                Accuracy : 0.7497
##                  95% CI : (0.7426, 0.7567)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : 0.99907
##
##                   Kappa : 0.3249
##
##  Mcnemar's Test P-Value : 0.00213
##
##             Sensitivity : 0.8271
##             Specificity : 0.5037
##          Pos Pred Value : 0.8412
```

```
##          Neg Pred Value : 0.4782
##              Prevalence : 0.7607
##          Detection Rate : 0.6292
##    Detection Prevalence : 0.7480
##       Balanced Accuracy : 0.6654
##
##        'Positive' Class : <=50K
##
```

The fifth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13
3: Decision tree	0.8187278	
4: Decision tree with CP (Complexity Parameter)	0.8232323	cp = 0.00416666666666667
5: Random forest	0.7497270	n of trees = 500, n of var = 3

2.3.3.6 Sixth model: Random Forest with Cross Validation

Cross validation is applied to the Random Forest model.

During the running, this model was saved in a file (train_rf_2.rds) and uploaded to GitHub. The model file is saved on the working directory and load it every time to save running time.

The code to generate the model, save the model as a file and load the file with the model is as follows:

```
# Random Forest with CV ========================================

beep()

# The code in comments might take from minutes to hours to run

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 10:00:56 CDT"
```

```
set.seed(6)

# train_rf_2 <- train(income ~ .,
#                     method = "Rborist",
#                     tuneGrid = data.frame(predFixed = 2, minNode = c(3, 50)),
#                     data = censusTrain)


# saveRDS(train_rf,"train_rf.rds")

# Loading model from file to save running time
train_rf_2 <- readRDS("train_rf_2.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-06-16 10:00:56 CDT"
```

Next is the Random Forest model with Cross Validation generated using 300 trees with 3 variables tried at each split.

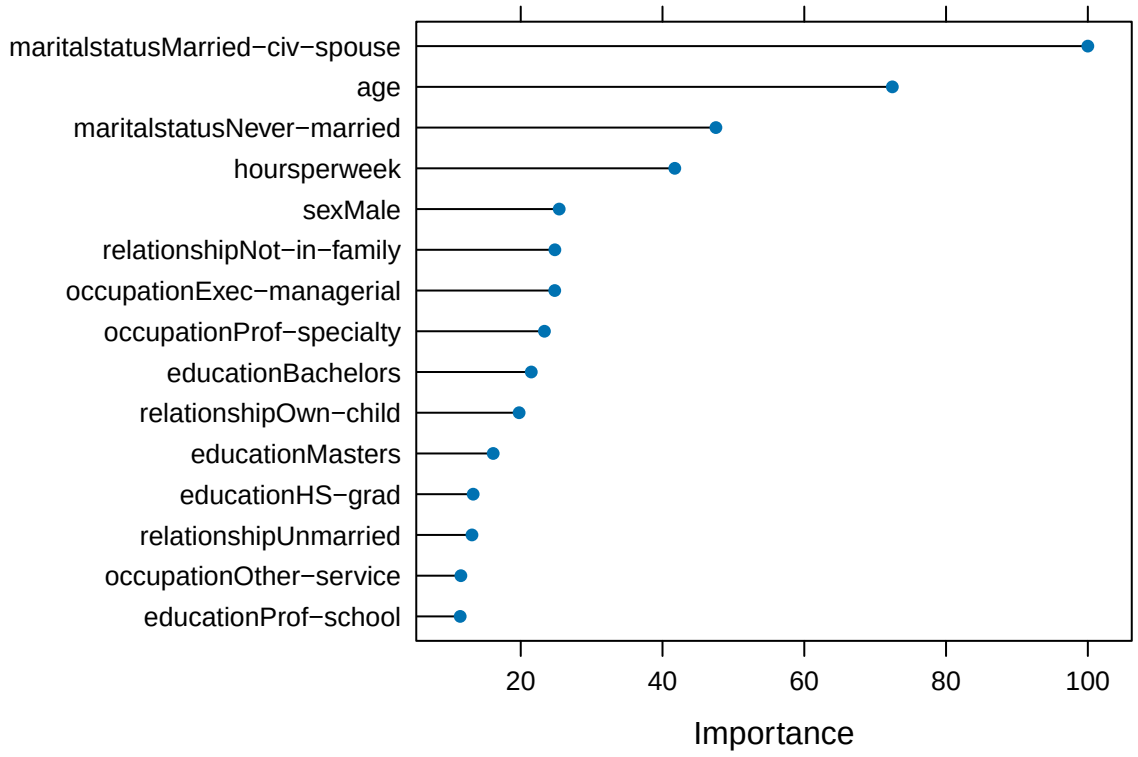
```
##
## Call:
##  randomForest(formula = income ~ ., data = censusTrain, ntree = 500,      do.trace = TRUE)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##         OOB estimate of  error rate: 16.4%
## Confusion matrix:
##       <=50K >50K class.error
## <=50K 23844 2165  0.08324042
## >50K   3443 4738  0.42085320
```

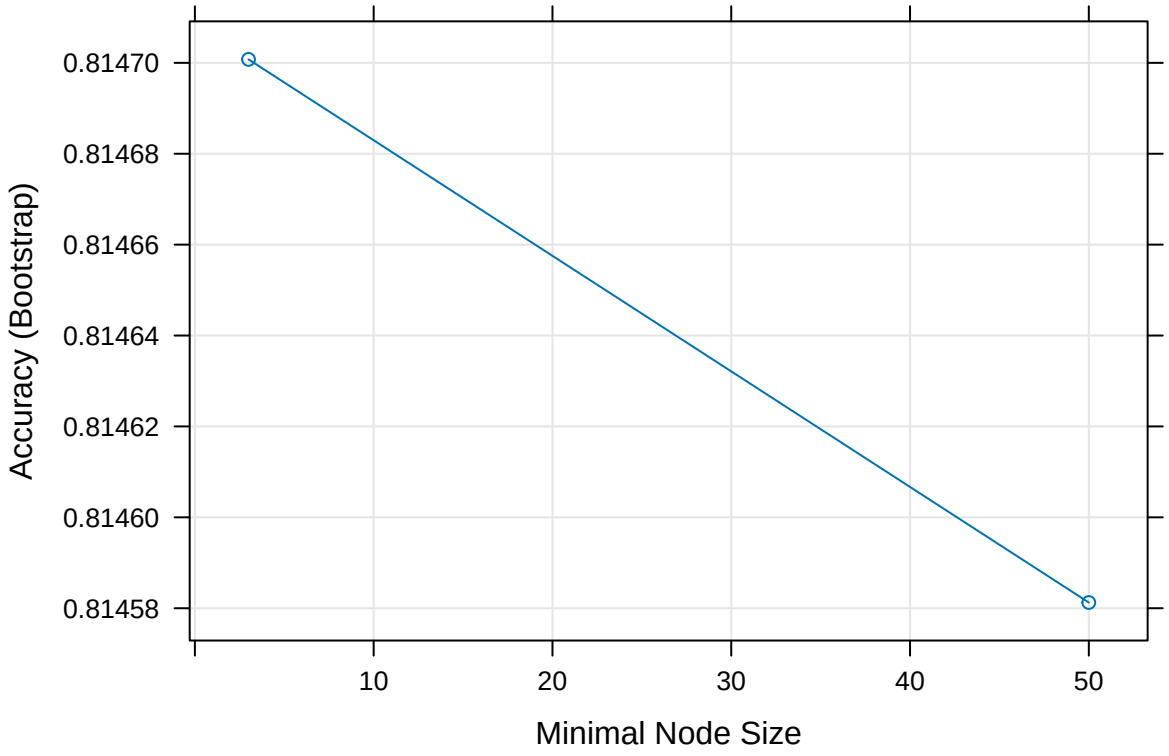
The Variables Importance plot is shown for the Random Forest CV model. This plot shows how much each variable contributes to the model's predictive power.

It shows that :

- MaritalStatus-Married-Civ-Spouse
- Age
- MaritalStatus-Never-married

Are the most important variables for this model.





The previous plot shows how Accuracy decreases as Minimal Node Size increases.

The following section shows the Confusion Matrix for this model. It shows:

- Accuracy : 0.816
- Sensitivity : 0.9775
- Specificity : 0.3026
- Balanced Accuracy : 0.6401

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K  >50K
##      <=50K 10895  2445
##      >50K    251  1061
##
##                Accuracy : 0.816
##                  95% CI : (0.8096, 0.8222)
##     No Information Rate : 0.7607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3566
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9775
##             Specificity : 0.3026
##          Pos Pred Value : 0.8167
```

```
##          Neg Pred Value : 0.8087
##              Prevalence : 0.7607
##          Detection Rate : 0.7436
##    Detection Prevalence : 0.9105
##       Balanced Accuracy : 0.6401
##
##        'Positive' Class : <=50K
##
```

The sixth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13
3: Decision tree	0.8187278	
4: Decision tree with CP (Complexity Parameter)	0.8232323	cp = 0.00416666666666667
5: Random forest	0.7497270	n of trees = 500, n of var = 3
6: Random forest with CV (Cross Valdidation)	0.8159978	

3 RESULTS

The following table shows all six models and their Accuracies:

method	Accuracy	Parameter
1: KNN	0.8099918	K = 21
2: KNN with CV (Cross Validation)	0.8101283	K = 13
3: Decision tree	0.8187278	
4: Decision tree with CP (Complexity Parameter)	0.8232323	cp = 0.00416666666666667
5: Random forest	0.7497270	n of trees = 500, n of var = 3
6: Random forest with CV (Cross Valdidation)	0.8159978	

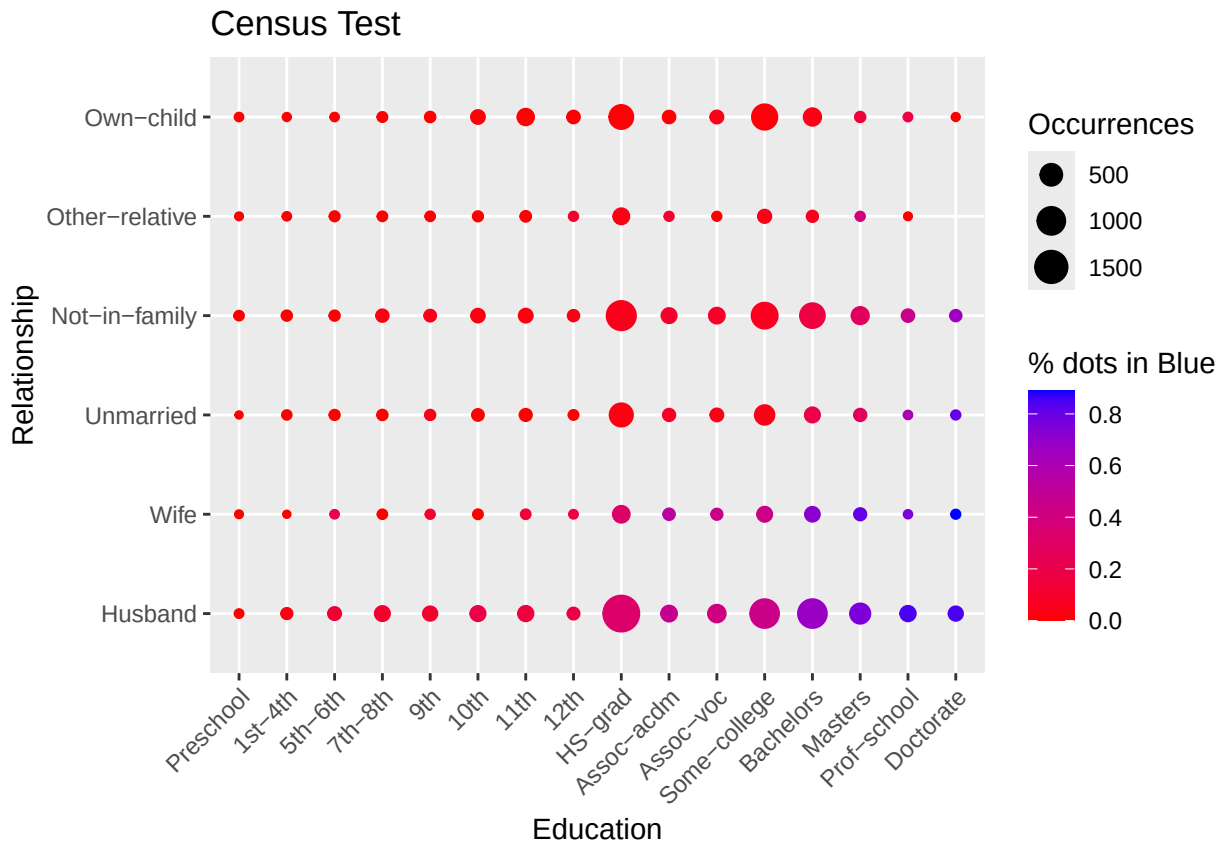
Accuracy has been improving when incorporating Cross Validation or the Complexity Parameter to the base models.

The Decision Tree model including the analysis of the Complexity Parameter optimized Accuracy among the six models. Accuracy was maximized through iteration.

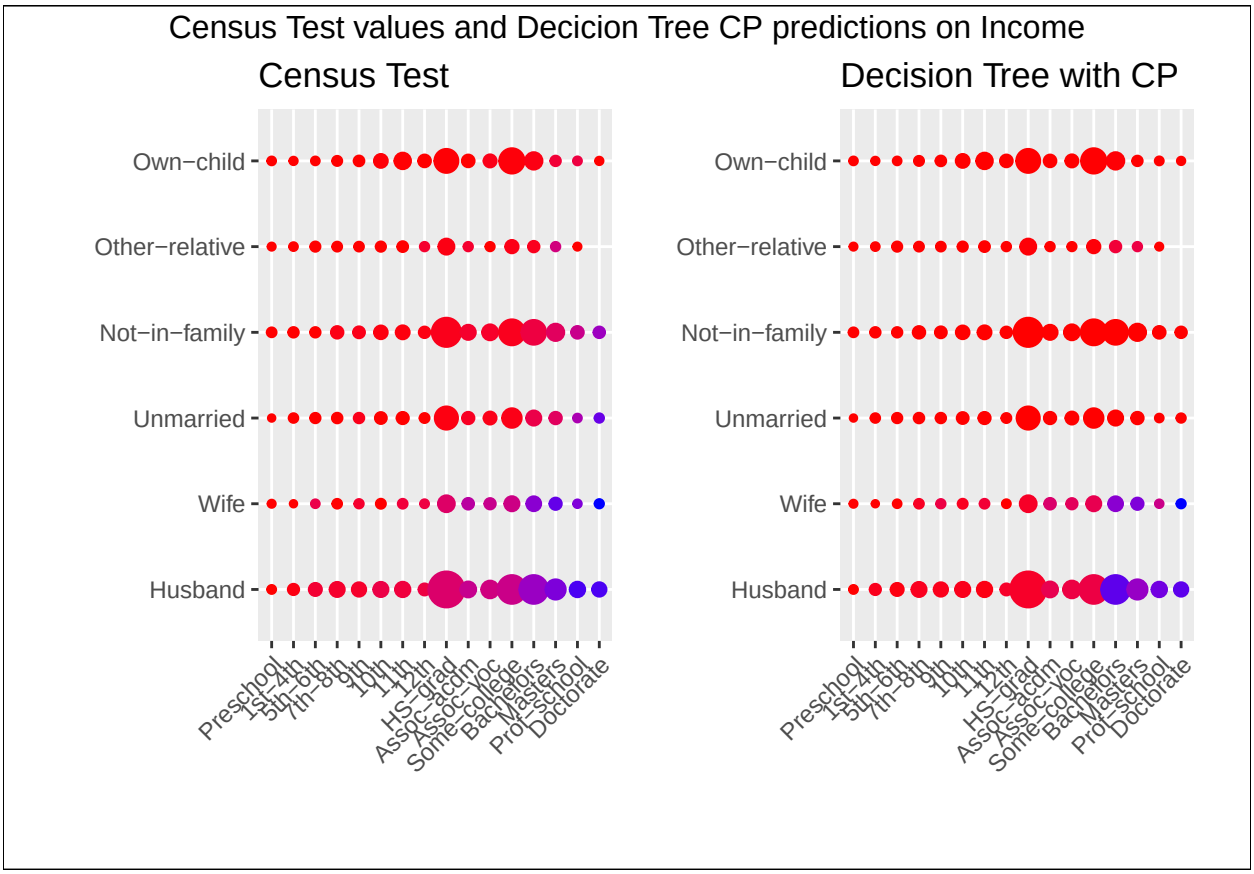
With the Decision Tree model with Complexity Parameter, an Income prediction model was built with an Accuracy of 82.32 percent.

The following graph shows the actual values of Income for the Census Test dataset. Because this is a two dimensional graph, the graph is illustrative and only two predictors out of the nine are shown, Education and Relationship.

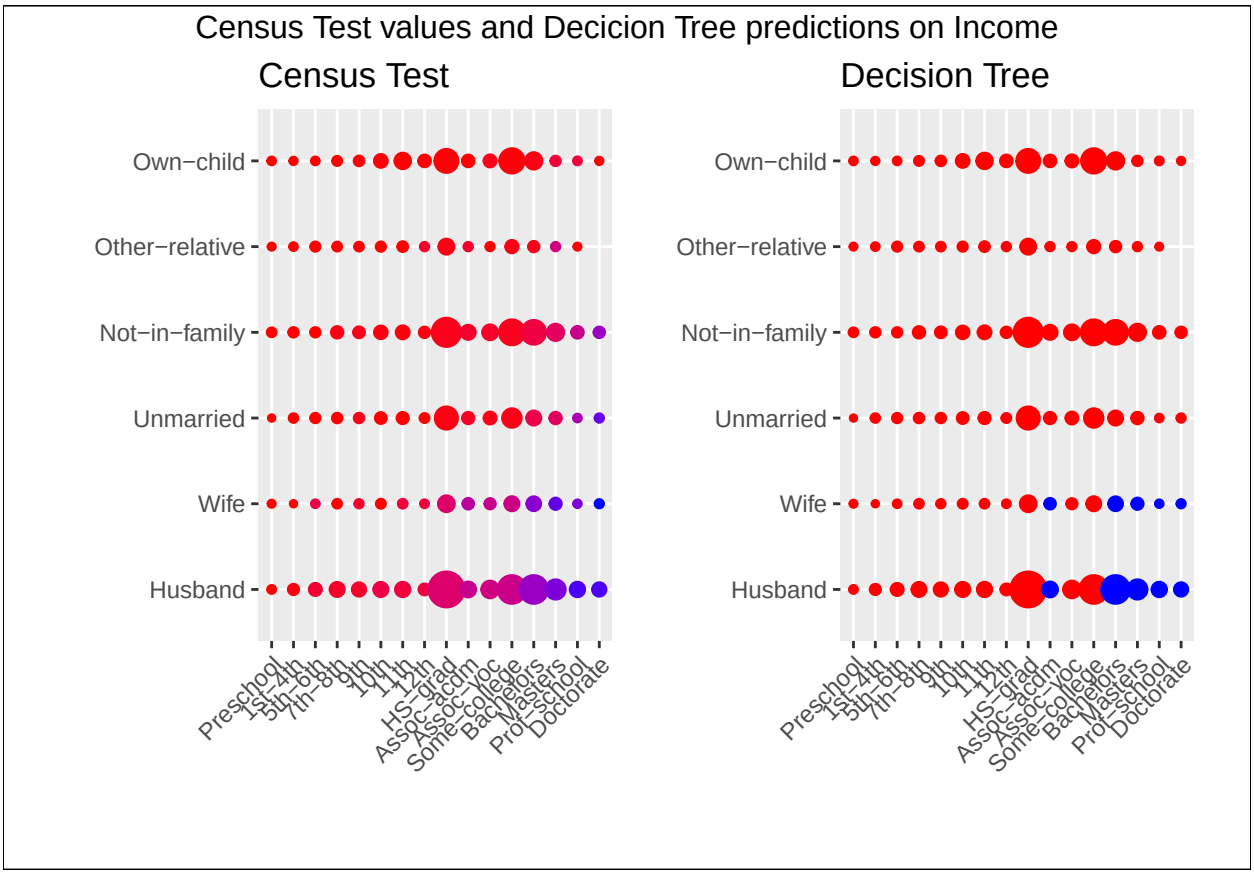
Every dot shows the frequency of occurrences that have "$>$50K" as income in blue as a percentage of the total number of records for that combination of Education and Relationship. The size of the dot represents the number of records for that combination of Education and Relationship.



The following graph shows an illustrative visual comparison between the actual values of the census test dataset and a graph generated with the predicted values of the model with the hightest Accuracy (Decision Tree with Complexity Parameter).



The following graph shows an illustrative visual comparison between the actual values of the census test dataset and a graph generated with the predicted values of the model with the second hightest Accuracy (Decision Tree).



4 CONCLUSION

In this analysis, six models were considered and accuracy was used to measure the predictive power of each model.

The models were increasing accuracy to get to the maximum when the analysis of Complexity Parameter was applied to the Decision Tree model.

Using Complexity Parameter, Accuracy was maximized through iteration.

With the Decision Tree model with Complexity Parameter, an Income prediction model was built with an Accuracy of 82.32 percent.

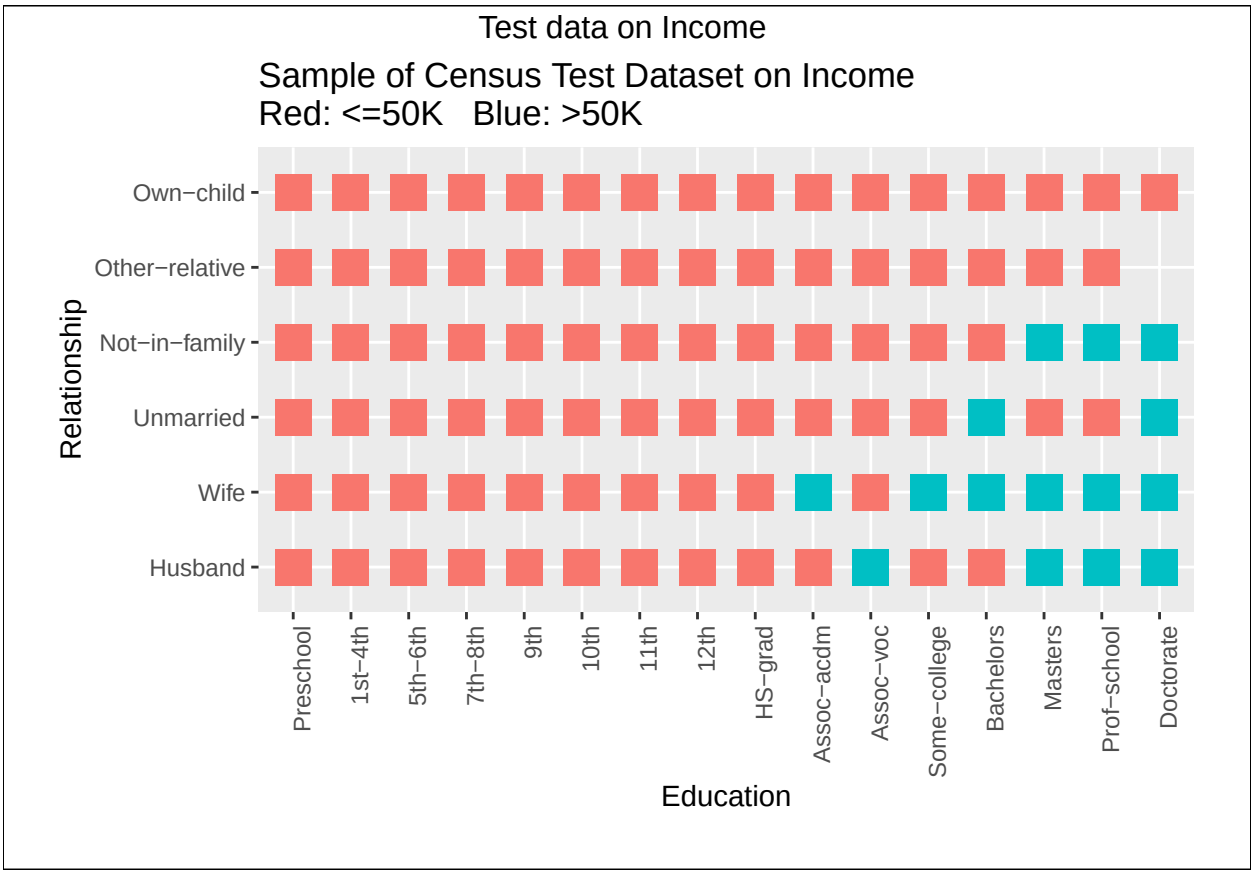
One potential limitation of the analysis is that the dataset has few records with income >50K in comparison to records with Income <=50K. This maybe a problem of Prevalence (when the ratio of Trues is close to 0 or 1).

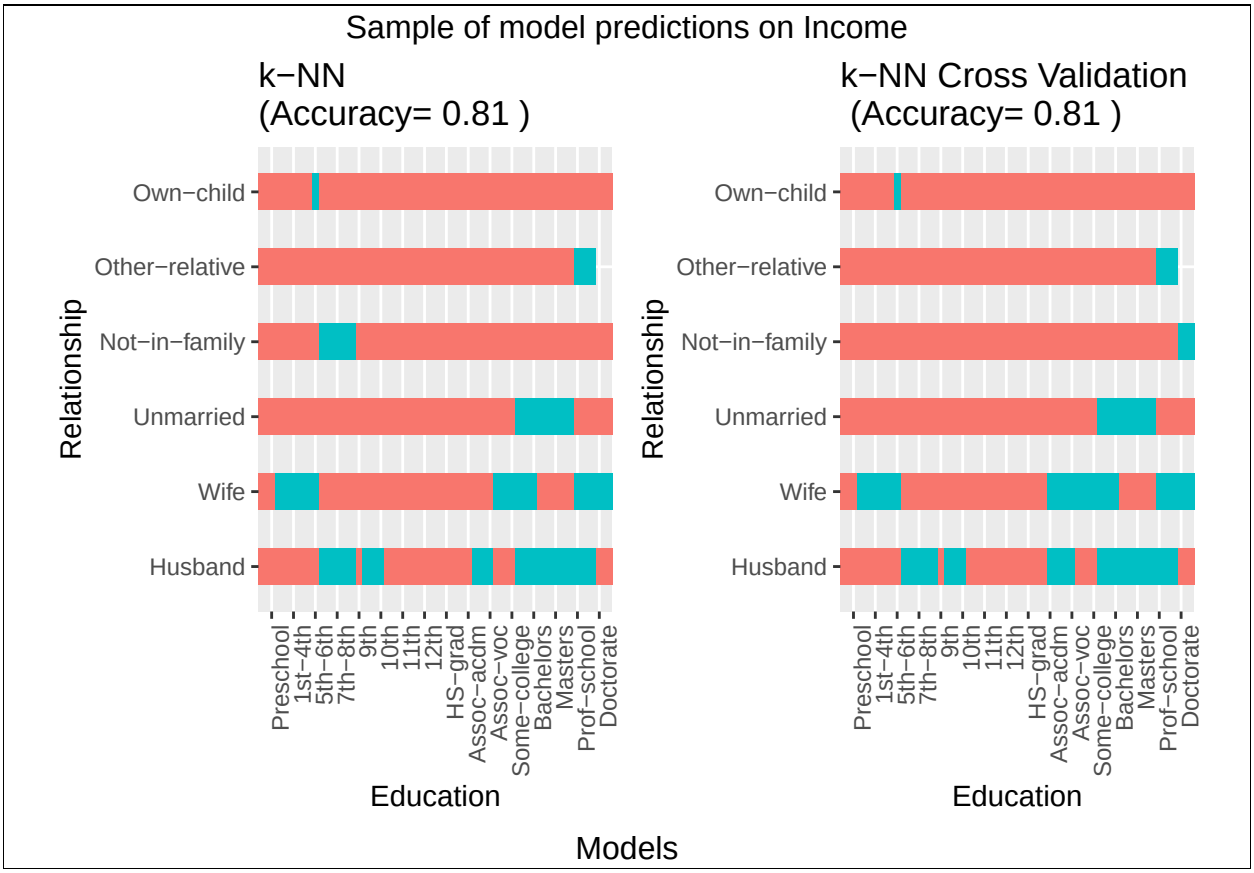
A couple of ways to address the issue of prevalence are:

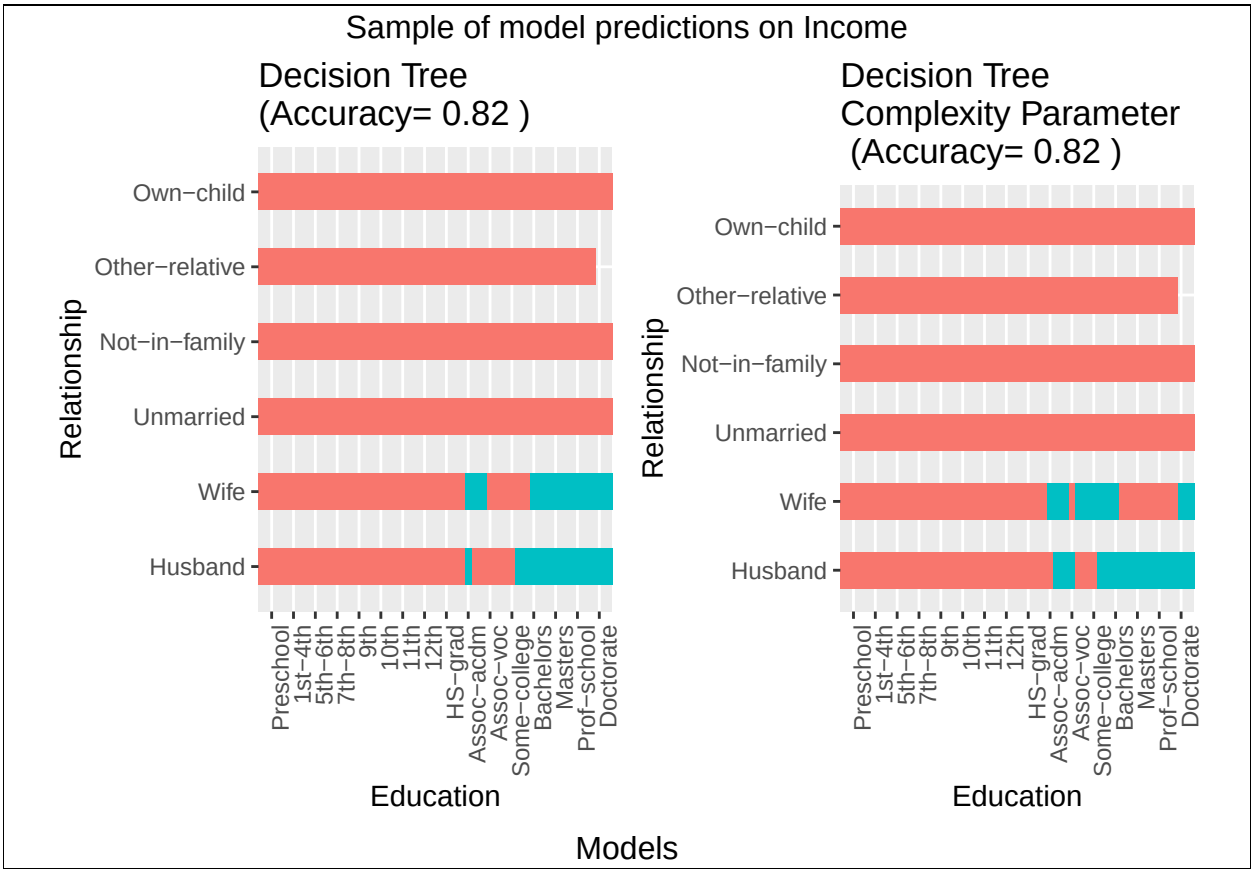
- generating a sample without prevalence
- analyzing sensitivity and specificity

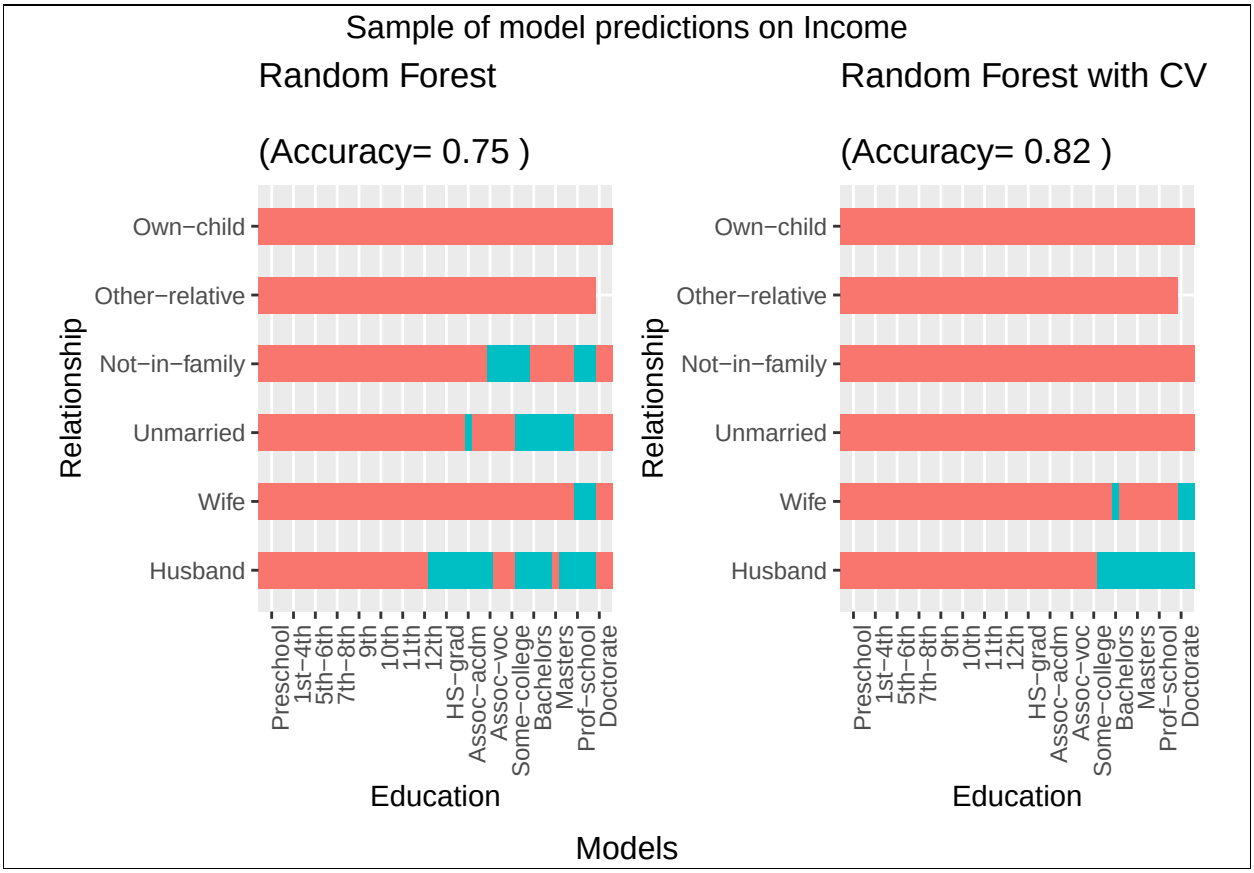
An additional step that may improve the prediction is to ensemble all six models into one. Ensembles combine multiple machine learning algorithms into one model to improve predictions (2).

5 ADDITIONAL GRAPHS




Sample of model predictions on Income
k−NN
(Accuracy= 0.81)
k−NN Cross Validation
(Accuracy= 0.81)
Relationship
Own−child
Other−relative
Not−in−family
Unmarried
Wife
Husband
Preschool
1st−4th
5th−6th
7th−8th
9th
10th
11th
12th
HS−grad
Assoc−acdm
Assoc−voc
Some−college
Bachelors
Masters
Prof−school
Doctorate
Education
Models


Sample of model predictions on Income
Decision Tree
(Accuracy= 0.82)
Decision Tree
Complexity Parameter
(Accuracy= 0.82)
Relationship
Own−child
Other−relative
Not−in−family
Unmarried
Wife
Husband
Preschool
1st−4th
5th−6th
7th−8th
9th
10th
11th
12th
HS−grad
Assoc−acdm
Assoc−voc
Some−college
Bachelors
Masters
Prof−school
Doctorate
Education
Models


Sample of model predictions on Income
Random Forest
(Accuracy= 0.75)
Random Forest with CV
(Accuracy= 0.82)
Relationship
Own−child
Other−relative
Not−in−family
Unmarried
Wife
Husband
Preschool
1st−4th
5th−6th
7th−8th
9th
10th
11th
12th
HS−grad
Assoc−acdm
Assoc−voc
Some−college
Bachelors
Masters
Prof−school
Doctorate
Education
Models

6 REFERENCES

1. Becker, B. & Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20.

2. Irizarry, R. Introduction to Data Science - Data Analysis and Prediction Algorithms with R (32.5 Ensembles) https://rafalab.dfci.harvard.edu/dsbook/machine-learning-in-practice.html#ensembles